Assignment 1

Code — A1Q1.m

```
clc
close all
clear all
format short
n = 19;
true_value = factorial(n)
app_value = (sqrt((2*n + 1/3)*pi))*(n^n)*(exp(-n))

%error
error = abs((true_value - app_value)/(true_value))
```

Output

```
true_value =

   1.2165e+17


app_value =

   1.2164e+17


error =

   1.8720e-05
```

Code — A1Q2.m

```
clc
close all
clear all

format short

a = 27;
b = 43;
c = 57;

disp("=======")
disp("[part a]")
disp("=======")
%ANGLE Y
ALPHA = acosd((a^2 - b^2 - c^2)/(-2*b*c))
BETA = acosd((b^2 - c^2 - a^2)/(-2*a*c))
GAMMA = acosd((c^2 - a^2 - b^2)/(-2*a*b))

disp("=======")
disp("[part b]")
disp("=======")
```

```
LHT = ( b - c ) / ( b + c )
RHT = (tand(1/2*(BETA - GAMMA))) / (tand(1/2*(BETA + GAMMA)))

tol = 0.00001;

if (LHT - RHT) < tol
    disp("RHT = LHT, thus the eqn holds")
else
    disp("RHT /= LHT, thus the eqn doesnt hold")
end
```

Output

```
=======
[part a]
=======

ALPHA =

   26.9669


BETA =

   46.2365


GAMMA =

  106.7966

=======
[part b]
=======

LHT =

   -0.1400


RHT =

   -0.1400

RHT = LHT, thus the eqn holds
```

Code — A1Q3.m

```
clc
close all
clear all

a = [5,8,-1,0,2];
b = [4,1,9,-2,3];
c = [-3,5,0,6,1];
```

```
disp("========")
disp("[Part a]")
disp("========")
vector_a = [a(:,1:3) b(:,1:3) c(:,1:3)]

disp("========")
disp("[Part b]")
disp("========")
%vector_b = [a(3:5) b(3:5) c(3:5)]'
vector_b = vector_a'

disp("========")
disp("[Part c]")
disp("========")
MAT_c = [a; b; c;]

disp("========")
disp("[Part d]")
disp("========")
MAT_d = [c(1:5); b(1:5); a(1:5);]'

disp("========")
disp("[Part e]")
disp("========")
MAT_e = [a(2:4); b(3:5); c(1:3);]
```

Output

```
========
[Part a]
========

vector_a =

     5     8    -1     4     1     9    -3     5     0

========
[Part b]
========

vector_b =

     5
     8
    -1
     4
     1
     9
    -3
     5
     0

========
[Part c]
========

MAT_c =

     5     8    -1     0     2
```

```
     4     1     9    -2     3
    -3     5     0     6     1

========
[Part d]
========

MAT_d =

    -3     4     5
     5     1     8
     0     9    -1
     6    -2     0
     1     3     2

========
[Part e]
========

MAT_e =

     8    -1     0
     9    -2     3
    -3     5     0
```

Code — A1Q4.m

```
clc
clear all
close all

format short

k = [0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75; 2, 4, 6, 8, 10, 12, 14; 25, 30, 35, 40, 45, 50, 55;]

disp("========")
disp("[Part A]")
disp("========")
row_1 = [k(1,1:2) k(1,6:7)];
row_2 = [k(3,1:2), k(3,6:7)];
MAT_a = [row_1; row_2;];
ka = MAT_a

disp("========")
disp("[Part B]")
disp("========")
row_a = [k(:,2)' k(:,5)'];
row_b = k(2, 1:6);
row_c = k(3, 2:7);

kb = [row_a; row_b; row_c;]
```

Output

```
k =

    0.2500    0.5000    0.7500    1.0000    1.2500    1.5000    1.7500
    2.0000    4.0000    6.0000    8.0000   10.0000   12.0000   14.0000
```

```
   25.0000   30.0000   35.0000   40.0000   45.0000   50.0000   55.0000

========
[Part A]
========

ka =

    0.2500    0.5000    1.5000    1.7500
   25.0000   30.0000   50.0000   55.0000

========
[Part B]
========

kb =

    0.5000    4.0000   30.0000    1.2500   10.0000   45.0000
    2.0000    4.0000    6.0000    8.0000   10.0000   12.0000
   30.0000   35.0000   40.0000   45.0000   50.0000   55.0000
```

Code — A1Q5.m

```
clc
close all
clear all

A = eye(2,2)
B = ones(1,2)
C = zeros(3,3)

D(1:3, 1:3) = C;
D(3, 4:5) = B;
D(1:2, 4:5) = A
```

Output

```
A =

     1     0
     0     1


B =

     1     1


C =

     0     0     0
     0     0     0
     0     0     0


D =
```

```
     0     0     0     1     0
     0     0     0     0     1
     0     0     0     1     1
```

Code — A1Q6.m

```
clc
close all
clear all

a = 0.75
b = 11.3

x = [2,5,1,9];
y = [0.2, 1.1, 1.8, 2]
z = [-3,2,5,4]

A = ( (x.^1.1 .*y.^-2 .* z.^5)/(a+b)^(b/3) + (a .*(z./x + y./2) ./z.^a) )
```

Output

```
a =

    0.7500


b =

   11.3000


y =

    0.2000    1.1000    1.8000    2.0000


z =

    -3     2     5     4


A =

  -0.7783 + 0.3257i   0.4368 + 0.0000i   1.4052 + 0.0000i   0.6263 + 0.0000i
```

Assignment 2

Code — A2Q1.m

```
clc
clear all
close all

format rational

v = [5 4 3 2]

disp("==================")
disp("    PART A        ")
disp("==================")

V = v + v;
a = 1 ./ V

disp("==================")
disp("    PART B        ")
disp("==================")

b = v.^v

disp("==================")
disp("    PART C        ")
disp("==================")

c = v./sqrt(v)

disp("==================")
disp("    PART D        ")
disp("==================")

d = (v.^2)./(v.^v)
```

Output

```
v =

       5              4              3              2

==================
    PART A
==================

a =

       1/10           1/8            1/6            1/4

==================
    PART B
==================

b =
```

```
    3125            256             27              4

==================
    PART C
==================

c =

    2889/1292         2           1351/780       1393/985

==================
    PART D
==================

d =

       1/125          1/16           1/3             1
```

Code — A2Q2.m

```
clc
close all
clear all

A = [25 40 60 70 32 0; 1 1 1 1 1 1; 0 -1 1 0 0 0; 0 1 0 1 -10 0; -1 1 1 0 0 0; 1 0 1 0 -4 -4]
%x = ['STUDENT'; 'ALUMNI'; 'FACULTY'; 'PUBLIC'; 'VETERANS'; 'G';]
b = [4897000; 100000; -11000; 0; 0; 0;]

x = round((A^-1)*b)



student = x(1)
alumni = x(2)
faculty = x(3)
public = x(4)
veterans = x(5)
guests = x(6)
```

Output

```
A =

    25    40    60    70    32     0
     1     1     1     1     1     1
     0    -1     1     0     0     0
     0     1     0     1   -10     0
    -1     1     1     0     0     0
     1     0     1     0    -4    -4


b =

     4897000
      100000
      -11000
           0
           0
           0
```

```
x =

       25000
       18000
        7000
       42000
        6000
        2000


student =

       25000


alumni =

       18000


faculty =

        7000


public =

       42000


veterans =

        6000


guests =

        2000
```

Code — A3Q3.m

```
clc
clear all
close all

format long
%format rational

disp("==================")
disp("    PART A        ")
disp("==================")

n = [1:5];
```

```
s = (n.^2)./(2.^n);
sum(s)

disp("==================")
disp("    PART B        ")
disp("==================")

n = [1:15];
s = (n.^2)./(2.^n);
sum(s)

disp("==================")
disp("    PART C        ")
disp("==================")

n = [1:30];
s = (n.^2)./(2.^n);
sum(s)
```

Output

```
==================
    PART A
==================

ans =

   4.406250000000000

==================
    PART B
==================

ans =

   5.991119384765625

==================
    PART C
==================

ans =

   5.999999044463038
```

Code — A2Q4.m

```
clc
close all
close all

format short

r = 14;
h_vals = [0:2:14];

%V = round(pi*r^2.*h_vals);
```

```
V = round(pi*r^2.*h_vals);
%round(V);

mat(1,:) = h_vals;
mat(2,:) = V;

MAT = mat';
disp("  HEIGHT   VOLUME")
disp(table(MAT))
```

Output

```
  HEIGHT   VOLUME
       MAT
    ___________

     0       0
     2    1232
     4    2463
     6    3695
     8    4926
    10    6158
    12    7389
    14    8621
```

Code — A2Q5.m

```
clc
close all
clear all

%size = input("Enter thy size of grade array ");
%size = 3
%grades = (1:size);

%for k = 1:1:size
%    %disp(k)
%    g = input("Enter value ");
%    grades(k) = g;
%end [93 77 51 62 99 41 82 77 71 68 100 46 78 80 83]
grades = input("Enter thy grades ")

len_values = (length(grades));
avg_values = (mean(grades));
std_values = (std(grades));

fprintf("\n")
fprintf('The are %5.2f grades; \n', len_values);
fprintf('The avg grade is %5.2f; \n', avg_values);
fprintf('Standard Deviation is %5.2f; \n', std_values);
```

Input

[93 77 51 62 99 41 82 77 71 68 100 46 78 80 83]

Output

grades =

  93 77 51 62 99 41 82 77 71 68 100 46 78 80 83

The are 15.00 grades;
The avg grade is 73.87;
Standard Deviation is 17.86;

Code — A2Q6.m

```
clc
clear
close all

T = [80 82 84 86 88 90 92 94];
R = [50 55 60 65 70 75]';

[t,r] = meshgrid(T,R);

HI = round(-42.379 + 2.04901523*T + 10.14333127*R - 0.22475541*(T.*R) - 6.83783e-3*(T.^2) -
5.481717e-2*(R.^2) + 1.22874e-3*(T.^2).*R + 8.5282e-4*T.*(R.^2) - 1.99e-6*(T.^2).*(R.^2));




disp("                            Temperature (F)   ")
%disp(T)
fprintf("%12i %5i %5i %5i %5i %5i %5i %5i %5i %5i \n", T)
fprintf("\n")
disp("Relative")
disp("Humidity")
disp("     (%)")
hi(:,2:9)=HI;
hi(:,1)=R;
disp(hi)
```

Output

```
                           Temperature (F)
          80    82    84    86    88    90    92    94
Relative
Humidity
     (%)
    50    81    83    85    88    91    95    99   103
    55    81    84    86    89    93    97   101   106
    60    82    84    88    91    95   100   105   110
    65    82    85    89    93    98   103   108   114
    70    83    86    90    95   100   106   112   119
    75    84    88    92    97   103   109   116   124
```

Assignment 3

Code — A3Q1.m

```
clc
close all
clear all

x = [0:0.01:10];
y = (x.^2).*exp(-x);
y1 = 2*x.*exp(-x) - (x.^2).*exp(-x);
y2 = 2*exp(-x) + (x.^2).*exp(-x) - 4.*x.*exp(-x);

idx = find(x==3)

plot(x,y, '-','linewidth',2,'markersize',13)
hold on;
plot(x(idx), y(idx), 'o', 'MarkerFaceColor','green', 'markersize', 10);
plot(x(idx), y1(idx), 'o', 'MarkerFaceColor','red', 'markersize', 11);
plot(x(idx), y2(idx), 'o', 'MarkerFaceColor','black', 'markersize', 13);
%p1 = plot(3,0.0003921,'o','MarkerFaceColor','red');

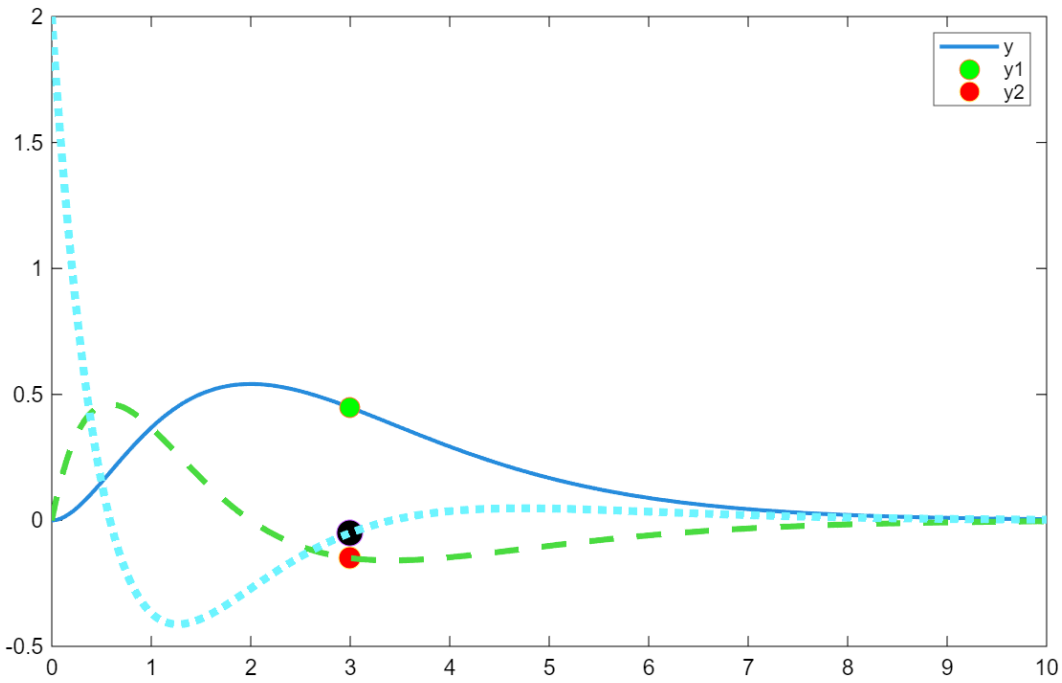


plot(x,y1,'--','linewidth',3,'markersize',14)
plot(x,y2,':','linewidth',4,'markersize',15)
legend("y","y1","y2")
```

Output

```
idx =

   301
```



Code — A3Q2.m

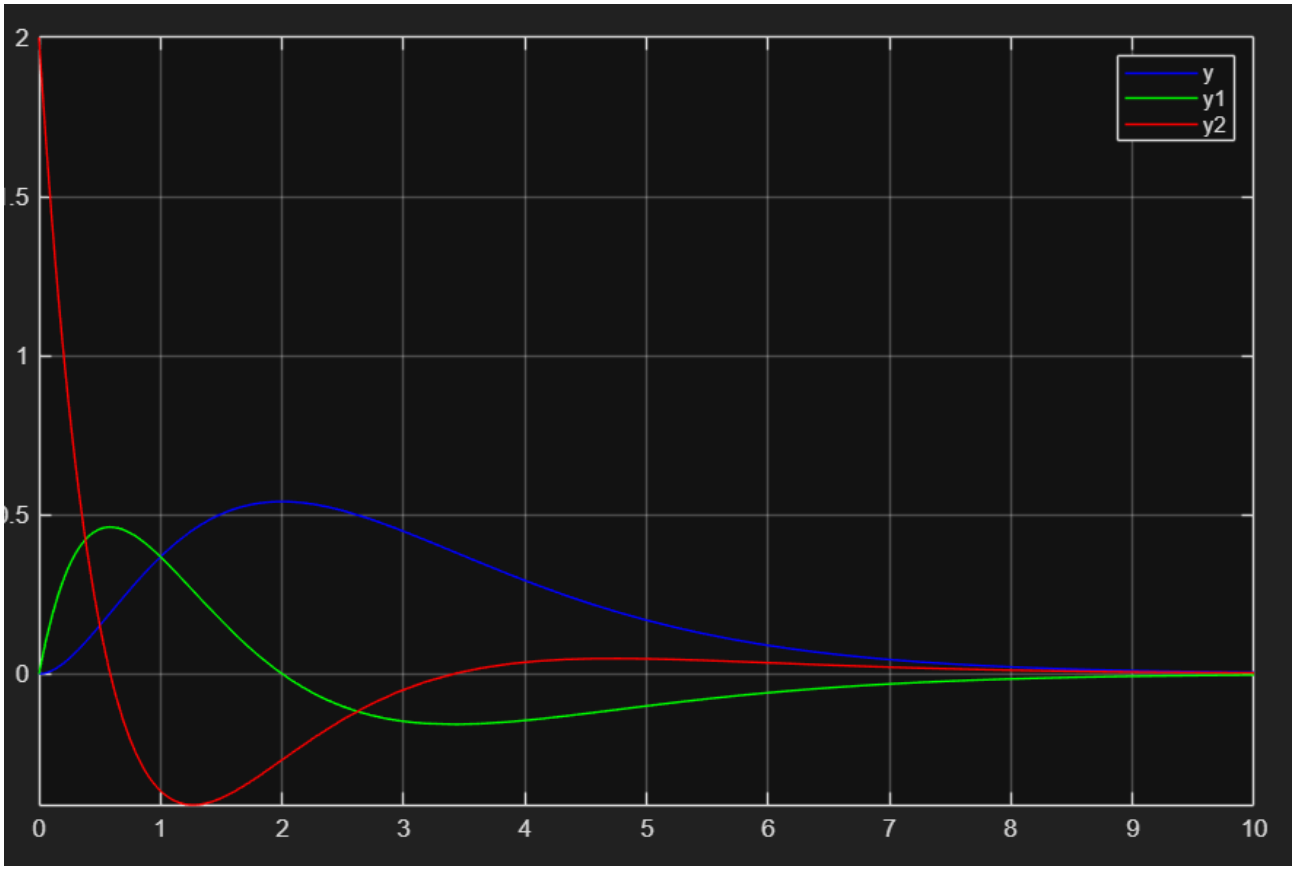
```
clc
close all
clear all

syms x
%x = [0:0.01:10];
y = @(x) (x.^2).*exp(-x);
%y1 = @(x) 2*x.*exp(-x) - (x.^2).*exp(-x);
%y2 = @(x) 2*exp(-x) + (x.^2).*exp(-x) - 4.*x.*exp(-x);

y1 = diff(y, x)
y2 = diff(y, x, 2)

fplot(y, [0 10],'b');
hold on;
fplot(y1, [0 10],'g');
fplot(y2, [0 10],'r');
legend("y","y1","y2")
```

Output



Code — A3Q3.m

```
%y = @(x) (x.^2 - 6.*x + 7)/(x.^3 - 8)

%fplot(y, [0 1.9], 'b');
%hold on;
%fplot(y, [2.1 4], 'b');

clc;
close all;
clear all;

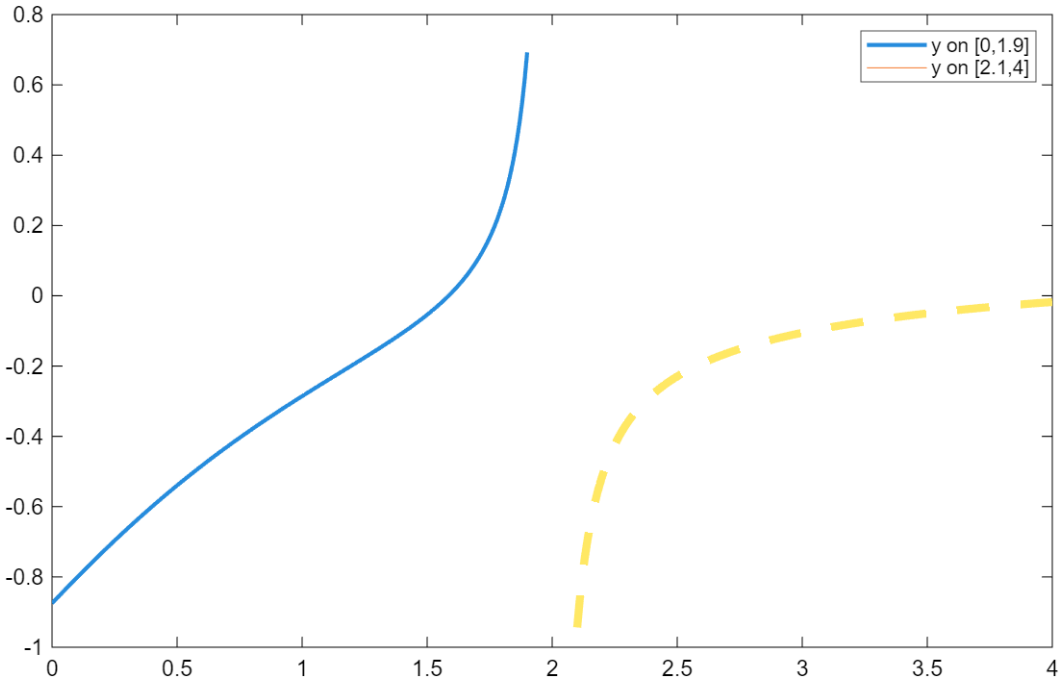
x1 = [0:0.01:1.9];
x2 = [2.1:0.01:4];

y1 = (x1.^2 - 6.*x1 + 7) ./ (x1.^3 - 8);
y2 = (x2.^2 - 6.*x2 + 7) ./ (x2.^3 - 8);

plot(x1, y1, '-', 'LineWidth', 2, 'MarkerSize', 13);
hold on;
plot(2, 0);
plot(x2, y2, '--', 'LineWidth', 4, 'MarkerSize', 14);

legend("y on [0,1.9]", "y on [2.1,4]");
%title('Plot of y = (x^2 - 6x + 7)/(x^3 - 8)');
%grid on;
```

Output



Code — A3Q4.m

```
clc;
close all;
clear all;

t = [10:10:70];
h = [9 22 44 63 80 94 97];

H = @(t) (100.8) ./ (1 + 23 .* exp(-0.093 .* t));

fplot(H)
hold on;
plot(t, h, 'o', 'LineWidth', 2, 'MarkerSize', 8)
%flpot(
%hold on;
%plot(t, H, '-', 'LineWidth', 2);

legend("Measured h", "Model H");
xlabel('t');
ylabel('h, H');
title('Comparison of Measured and Model Values');
grid on;
```

Output



Code — A3Q5.m

```
clc;
close all;
clear all;

% Symbolic work
syms t
y = 8 - 4*t^3*exp(-0.4*t) + 2*t^2;
y1 = diff(y,t);   % first derivative
y2 = diff(y,t,2); % second derivative

% Numeric evaluation
x = [0:0.1:8];
y1_num = double(subs(y1,t,x));
y2_num = double(subs(y2,t,x));



% Plotting
subplot(3,1,1);
%plot(x, double(subs(y,t,x)), 'LineWidth', 2);
fplot(y,[0,8])
title('Position x(t)');
xlabel('t'); ylabel('y(t)');

subplot(3,1,2);
%plot(x, y1_num, 'LineWidth', 2);
fplot(y1, [0,8])
title('Velocity y1(t)');
xlabel('t'); ylabel('y1(t)');

subplot(3,1,3);
%plot(x, y2_num, 'LineWidth', 2);
fplot(y2, [0,8])
title('Acceleration y2(t)');
xlabel('t'); ylabel('y2(t)');
```

Output

```
ERROR: Undefined function 'syms' for input arguments of type 'char'.
```

Code — A3A6.m

```
clc
close all
clear all

figure(1)

fimplicit(@(x,y) x.^(2).^(1/3) + y.^(2).^(1/3) - 1)
title("Cartesian Equation")



figure(2)

x = @(t) (cos(t)).^3
y = @(t) (sin(t)).^3

%fplot(x,y, [-pi pi])
```

```
fplot(x, y, [-pi pi], 'r');
title("Parametric Equation")
%legend("Implicit curve", "Parametric curve");
%xlabel('x');
%ylabel('y');
```

Output

```
x =

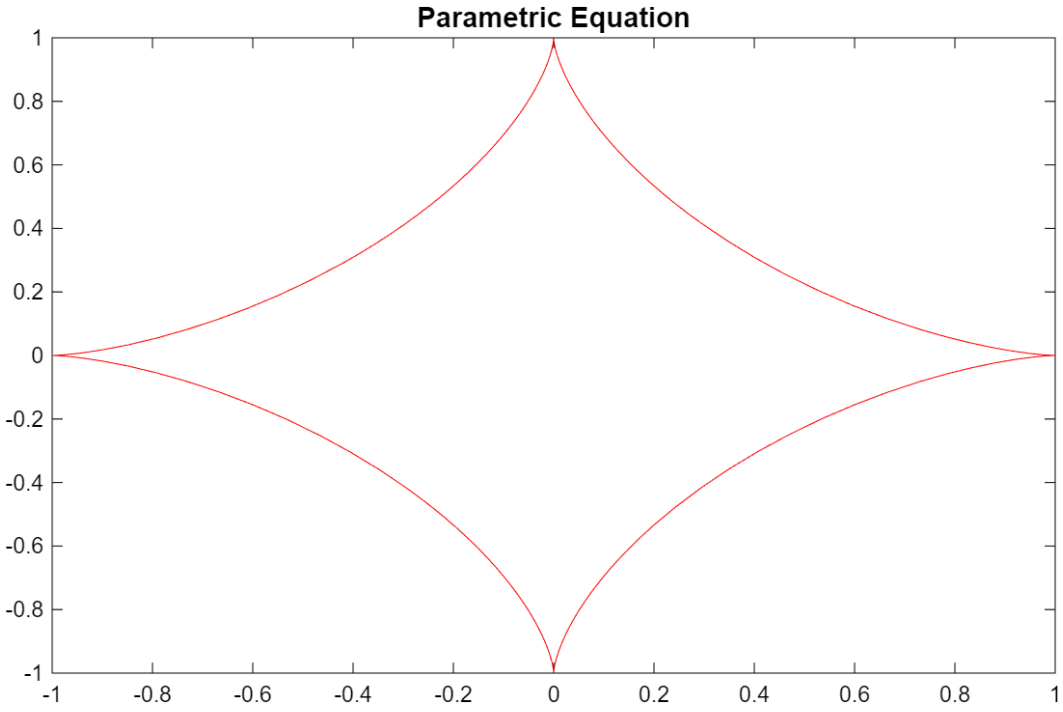
  function_handle with value:

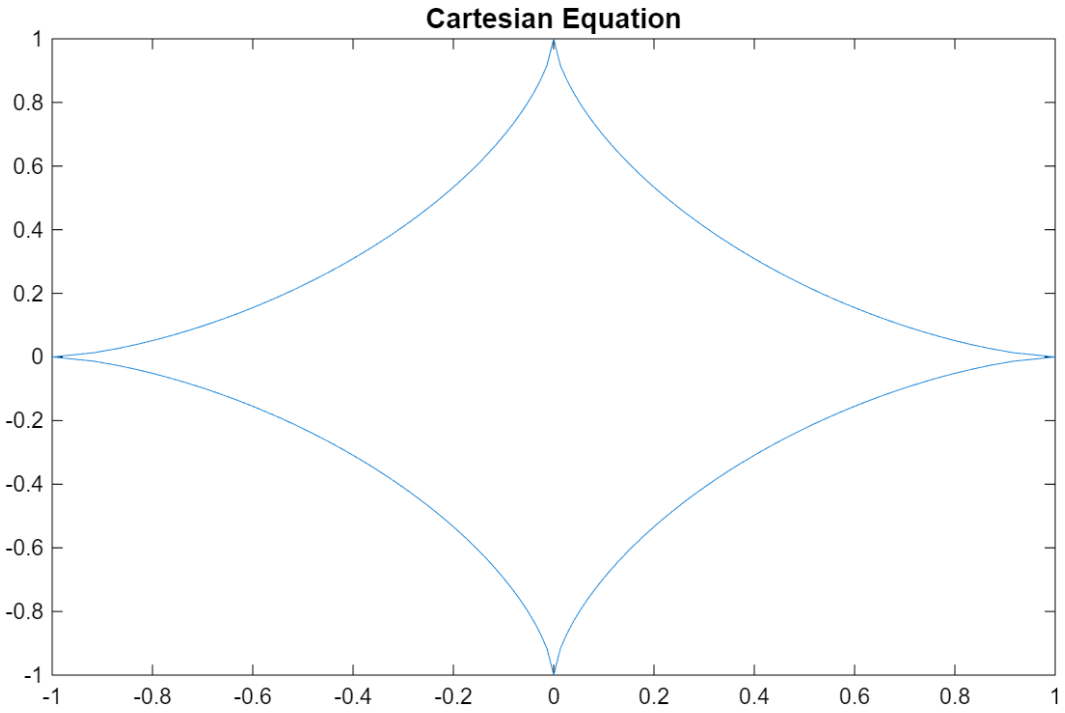
    @(t)(cos(t)).^3


y =

  function_handle with value:

    @(t)(sin(t)).^3
```





Code — A3Q7.m

```
clc
close all
clear all

x = [1200, 3500, 3200, 1500, 1700];
di = {'Dhaka' 'Sylhet' 'Chattogang' 'Rajshahi' 'Khulna'}

explode = [0 1 0 0 0];

pie(x,explode)

legend(di)

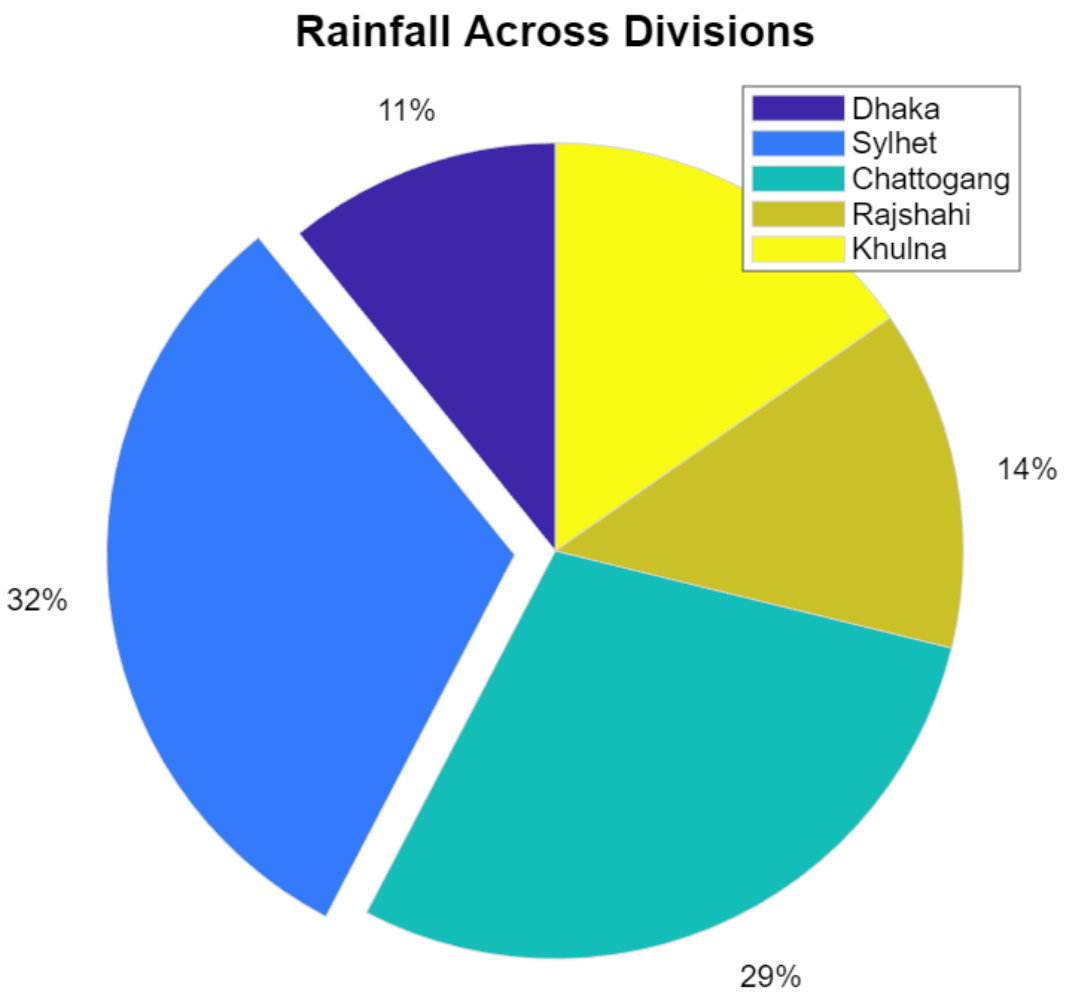
title("Rainfall Across Divisions")
```

Output

```
di =

  1×5 cell array

    {'Dhaka'}    {'Sylhet'}    {'Chattogang'}    {'Rajshahi'}    {'Khulna'}
```



Code — A3Q8.m

```
clc
close all
clear all

MONTHS = categorical({'Jan' 'Feb' 'Mar' 'Apr' 'May' 'Jun' 'Jul' 'Aug' 'Sep' 'Oct' 'Nov' 'Dec'})

TEMP = [18 21 26 29 30 31 30 30 29 27 24 20];

b = bar(TEMP)
xticklabels(MONTHS)

b.FaceColor = "flat";

b.CData(3:6, :) = repmat([1 0 0], 4,1);
b.CData(11:12, :) = repmat([0 1 1], 2,1);
b.CData(1:2, :) = repmat([0 1 1], 2,1);

hold on
```

```
summer = bar(NaN, 'FaceColor', [1 0 0]);
winter = bar(NaN, 'FaceColor', [0 1 1]);
title("Monthly avg temp")
legend('Winter', 'Summer')
%legend({'Line 1','Line 2'})
%legend({'cos(x)','cos(2x)','cos(3x)','cos(4x)'},'Location','northwest','NumColumns',2)
%legend([summer, winter], {'Summer', 'Winter'});
```

Output

```
MONTHS =

  1×12 categorical array

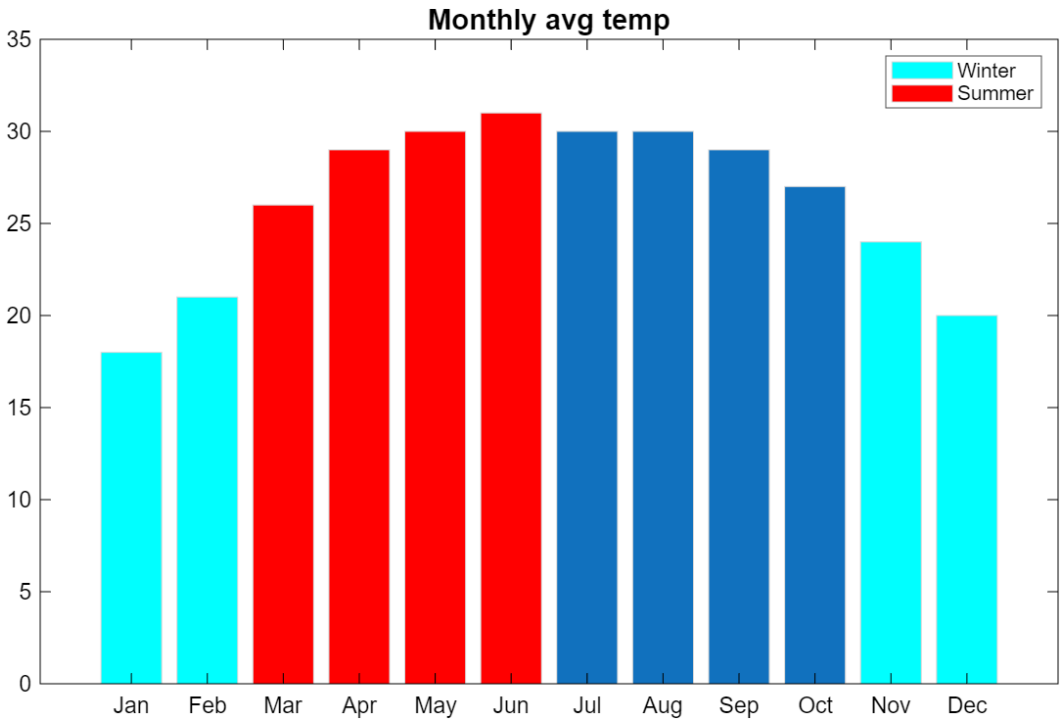
    Jan      Feb      Mar      Apr      May      Jun      Jul      Aug      Sep      Oct      Nov      Dec


b =

  Bar with properties:

    BarLayout: 'grouped'
     BarWidth: 0.8000
    FaceColor: [0.0660 0.4430 0.7450]
    EdgeColor: [0.1294 0.1294 0.1294]
    BaseValue: 0
        XData: [1 2 3 4 5 6 7 8 9 10 11 12]
        YData: [18 21 26 29 30 31 30 30 29 27 24 20]

  Use GET to show all properties
```



Code — A3Q9.m

```
clc
close all
clear all

Day = [1 2 3 4 5 6 7 8 9 10];
Max_Temp = [31 32 34 33 35 36 34 33 32 31];

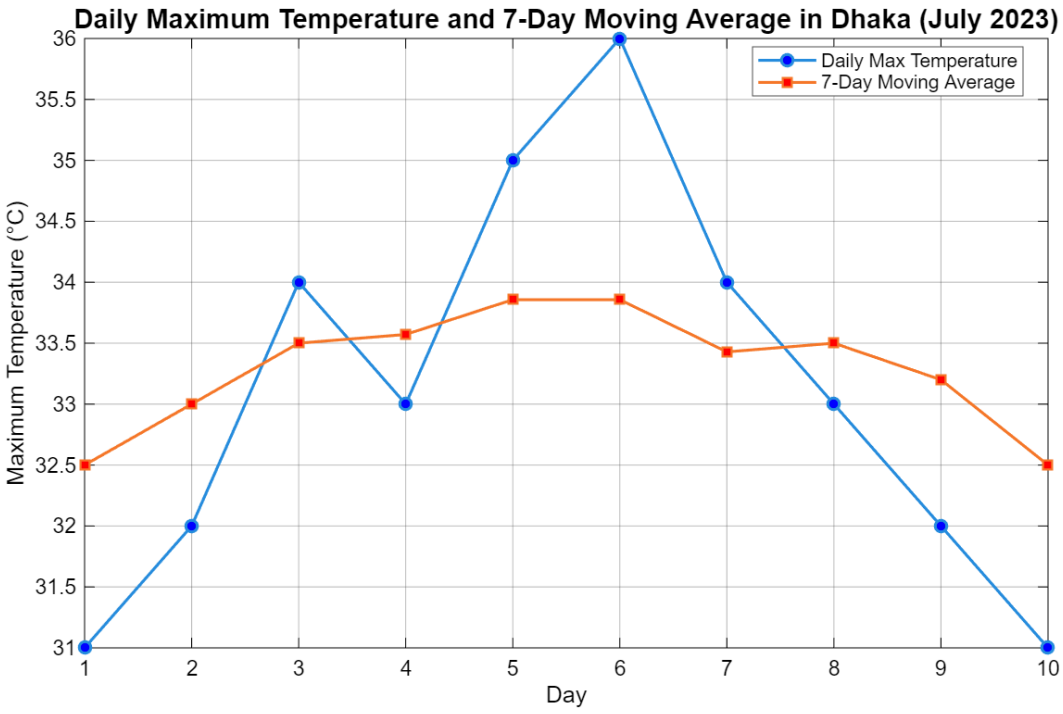

Moving_Avg = movmean(Max_Temp, 7);

figure;
plot(Day, Max_Temp, '-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot(Day, Moving_Avg, '-s', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
hold off;



title('Daily Maximum Temperature and 7-Day Moving Average in Dhaka (July 2023)');
xlabel('Day');
ylabel('Maximum Temperature (°C)');
legend('Daily Max Temperature', '7-Day Moving Average', 'Location', 'best');
grid on;
```

Output



Assignment 4

Code — A4Q1.m

```
clc
close all
clear all

m = 4;
n = 6;
Z = disp_prog(m,n)

function A = disp_prog(m,n)
    A = zeros(m,n);

    for i = 1:m
       for j = 1:n
            A(i,j) = 2*i - 3*j;
       end
    end
end
```

Output

```
Z =

    -1    -4    -7   -10   -13   -16
     1    -2    -5    -8   -11   -14
     3     0    -3    -6    -9   -12
     5     2    -1    -4    -7   -10
```

Code — A4Q2.m

```
clc
close all
clear all

%disp("Original Vector")
Z = input("Enter the vectos: \n");
X=disp_prog(Z);
disp("Modified Vectors")
disp(X)

function [A] = disp_prog(Z)

    n = length(Z);

    %disp("OG Vector")
    %disp(A)
    %fprintf("\n")
    for i = 1:n
        if Z(i) > 0
            A(i) = 2 * Z(i);
        else
            A(i) = 3 * Z(i);
        end
    end
end
```

Input

[1, -2, 3, -4, 5]

Output

```
Modified Vectors
     2    -6     6   -12    10
```

Code — A4Q3.m

```
clc
close all
clear all
format bank

NYC = [33 33 18 29 40 55 19 22 32 37 58 54 51 52 45 41 45 39 36 45 33 18 19 19 28 34 44 21 23 30
39];
DEN = [39 48 61 39 14 37 43 38 46 39 55 46 46 39 54 45 52 52 62 45 62 40 25 57 60 57 20 32 50 48
28];

avg_nyc = (sum(NYC)/length(NYC));
avg_den = (sum(DEN)/length(DEN));

disp("==========part a====================")
fprintf("The avg temp of NYC %0.2f \n",avg_nyc);
fprintf("The avg temp of DEN %0.2f \n",avg_den);

disp("==========part b====================")
cnt_nyc = count(NYC, DEN, avg_nyc, "A");
cnt_den = count(NYC, DEN, avg_den, "A");

fprintf("Desired num of days in NYC %d \n",cnt_nyc);
fprintf("Desired num of days in DEN %d \n",cnt_den);

disp("==========part c====================")
cnt_days = count(NYC, DEN, 0, "B");

fprintf("The number of days: %d\n", cnt_days)

function cnt = count(arr1, arr2, threshold, param)
    cnt = 0;

    if param == "A"
        for i = 1:length(arr1)
            if arr1(i) > threshold
                cnt = cnt + 1;
            end
        end
    end

    if param == "B"
        for i = 1:length(arr1)
            if arr1(i) < arr2(i)
                cnt = cnt + 1;
            end
        end
    end
end
```

```
%function cnt = count_above(arr, threshold)
%    cnt = 0;
%    for i = 1:length(arr)
%        if arr(i) > threshold
%            cnt = cnt + 1;
%        end
%    end
%end

%function cnt = count_less_than(arr1, arr2)
%    cnt = 0;
%    for i = 1:length(arr1)
%        if arr1(i) < arr2(i)
%            cnt = cnt + 1;
%        end
%    end
%end
```

Output

```
==========part a====================
The avg temp of NYC 35.23
The avg temp of DEN 44.48
==========part b====================
Desired num of days in NYC 15
Desired num of days in DEN 8
==========part c====================
The number of days: 22
```

Code — A4Q4.m

```
clc
close all
clear all

n = 4;
fprintf("Pascals trig for n = %d\n",n)
P4 = Pascals_trig(n);

%disp(P4);

n = 9;
fprintf("Pascals trig for n = %d\n",n)
P7 = Pascals_trig(n);

%disp(P7);

function p = Pascals_trig(n)
    p = zeros(n, n);

    for i = 1:n
        for j = 1:i
            p(i, j) = factorial(i - 1) / (factorial(j - 1) * factorial(i - j));
            fprintf('%5d',p(i,j))
        end
        fprintf("\n")
    end
    fprintf("\n")
```

```
end
```

Output

```
Pascals trig for n = 4
    1
    1    1
    1    2    1
    1    3    3    1

Pascals trig for n = 9
    1
    1    1
    1    2    1
    1    3    3    1
    1    4    6    4    1
    1    5   10   10    5    1
    1    6   15   20   15    6    1
    1    7   21   35   35   21    7    1
    1    8   28   56   70   56   28    8    1
```

Code — A4Q5.m

```
clc
close all
clear
format long

n_values = [5, 10, 40, 45, 55];

print_prog(n_values)


function p = print_prog(n_values)

    for k = 1:length(n_values)
        n = n_values(k);
        PI = estimate_pi_cubic(n);
        fprintf("The estimation of pi for n = %d: %.6f; error(rel) = %.6f\n", n, PI, abs(PI -
pi)/pi);
    end
end

function PI = estimate_pi_cubic(n)
    sum_val = 0;

    for i = 0:(n-1)
        sum_val = sum_val + ( (-1)^i / (2*i + 1)^3 );
    end

    PI = (32 * sum_val)^(1/3);
end
```

Output

```
The estimation of pi for n = 5: 3.142104; error(rel) = 0.000163
The estimation of pi for n = 10: 3.141526; error(rel) = 0.000021
The estimation of pi for n = 40: 3.141592; error(rel) = 0.000000
The estimation of pi for n = 45: 3.141593; error(rel) = 0.000000
The estimation of pi for n = 55: 3.141593; error(rel) = 0.000000
```

Code — A4Q6.m

```
clc;
close all;
clear;

N = 10000;
[z] = Mersp(N)

length(z)

function v = Mersp(N)
    cnt = 1;
    for i = 1:N
        p(i) = 2^i - 1;

        if isprime(p(i))

            %temp = p(i)
            %fprintf("%dth mersone prime is %d", i, p(i))
            v(cnt) = p(i);
            cnt = cnt + 1;
        end

        if p(i) > 10000
            break;
        end

    end
    %length(v)
end
```

Output

```
z =

           3           7          31         127        8191


ans =

     5
```

Code — A4Q7.m

```
clc
close all
clear all

disp("===========part a====================")
P1 = 8;
ans_a = halley(P1);
fprintf("The cubic root of 800 is: %f\n", ans_a);

disp("===========part b====================")
P2 = 59071;
ans_b = halley(P2);
fprintf("The cubic root of 59071 is: %f\n", ans_b);

function x = halley(P)
    x = 10; %initial guess of cubic root of P
    E = 1; %error term

    while E > 0.00001
        x_next = x * (x^3 + 2*P) / (2*x^3 + P);
        E = abs((x_next - x) / x);
        x = x_next;
    end
end
```

Output

```
===========part a====================
The cubic root of 800 is: 2.000000
===========part b====================
The cubic root of 59071 is: 38.945574
```

Code — A4Q8.m

```
clc
close all
clear all

disp("=========== Part (a) =============")
hw_a = [65, 79, 80, 50, 71, 73, 61, 70, 69, 74];
mid_a = [83, 91];
final_a = 84;
scale_factor = 80;

grade_a = compute_course_grade(hw_a, mid_a, final_a, scale_factor);
fprintf("Overall course grade for case (a): %.2f\n", grade_a);

disp("=========== Part (b) =============")
hw_b = [70, 69, 83, 45, 90, 89, 52, 78, 100, 87];
mid_b = [87, 72];
final_b = 90;
scale_factor = 100;

grade_b = compute_course_grade(hw_b, mid_b, final_b, scale_factor);
fprintf("Overall course grade for case (b): %.2f\n", grade_b);

function course_grade = compute_course_grade(hw, midterms, final_exam, scale_factor)
```

```
    sorted_hw = sort(hw);
    higher_hw = sorted_hw(3:end);
    avg_hw = mean(higher_hw);

    hw_score = (avg_hw / scale_factor) * 20;
    hw_component = hw_score;

    avg_mid = mean(midterms);

    if avg_mid >= final_exam
        mid_component = avg_mid * 0.40;
        final_component = final_exam * 0.40;
    else
        mid_component = avg_mid * 0.30;
        final_component = final_exam * 0.50;
    end

    course_grade = hw_component + mid_component + final_component;
end
```

Output

```
========== Part (a) ==========
Overall course grade for case (a): 86.56
========== Part (b) ==========
Overall course grade for case (b): 85.50
```

Assignment 5

Code — A5Q1.m

```
clc
close all
clear all

a = -pi;
b = pi;
tol = 10^(-5);
i = 0;
rel = 1;
f_ = @(x) x .* cos(x) - 2*x.^2 + 3*x - 1;
%f_ = @(x) 1/2 + 1/4*x^2 - x*sin(x) - 1/2*cos(2*x);
exact_val = fzero(f_, 0);

fplot(f_, [-2, 2])

[i,a,b,tol,rel] = MAIN(i,a,b,tol,rel, exact_val);

function [i,a,b,tol,rel] = MAIN(i,a,b,tol,rel, exact_val)
    fprintf("i  a         b       c        f(c)     Rel\n")

    if (A_FUNC(a) * A_FUNC(b)) < 0
        while rel > tol
            i = i + 1;
            t_old = a;
            t = (a+b)/2;

            if A_FUNC(t) == 0
               fprintf("Root found %0.2f\n", t)
               break;
            end

            rel = abs(exact_val - t_old) / exact_val;

            fprintf("%d  %0.5f  %0.5f  %0.5f  %0.5f  %f \n", i, a, b, t, A_FUNC(t), rel)

            if A_FUNC(t) * A_FUNC(a) < 0
               b = t;
            else
               a = t;
            end

        end
    else
        fprintf("Root doesnt exist between %f and %f\n\n", a, b)
    end
   % fprintf("Ans = %f\n", t)
end



function f = A_FUNC(x)
    f = 1/2 + 1/4*x^2 - x*sin(x) - 1/2*cos(2*x);
end

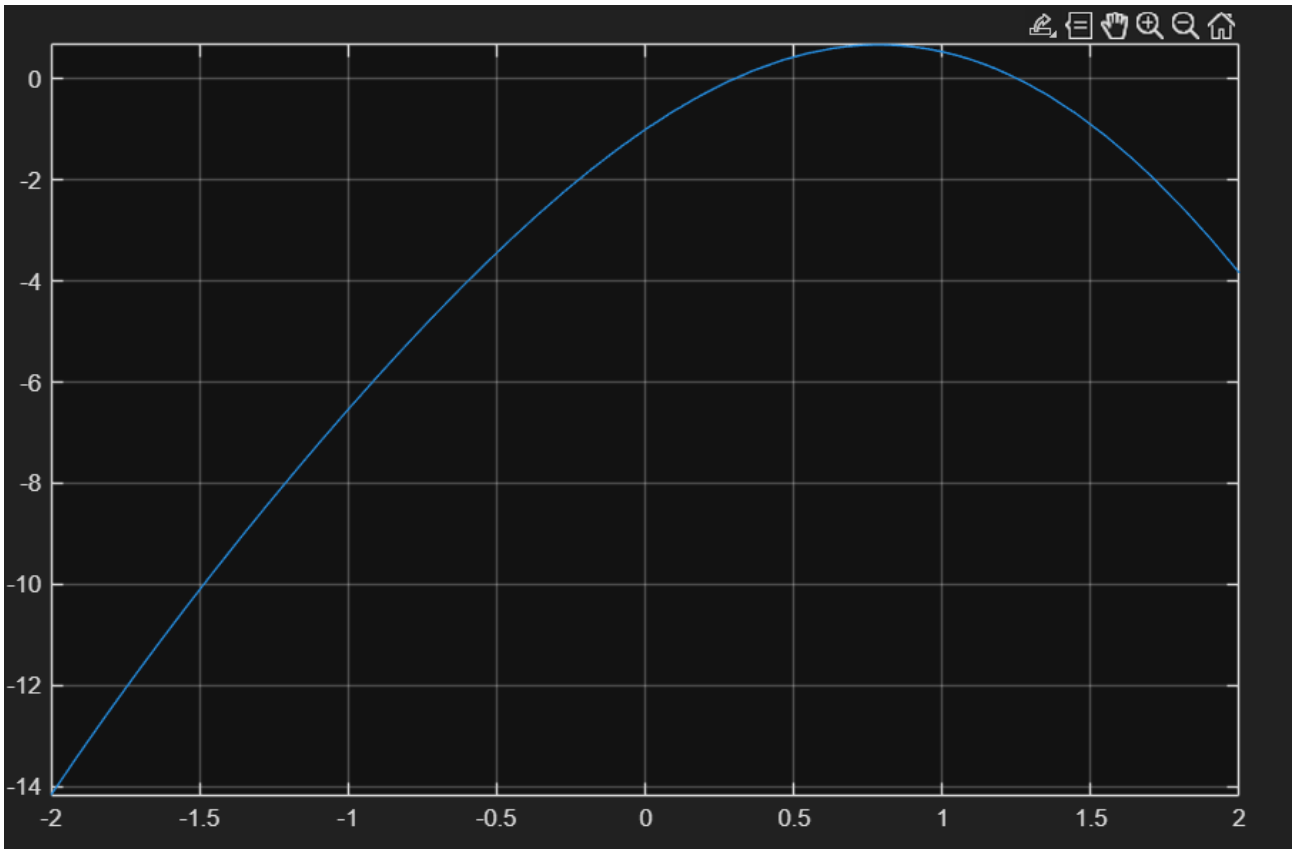
%function f = A_FUNC(x)
```

```
%    f = x * cos(x) - 2*x^2 + 3*x - 1;
%end
```

Output

```
i  a         b       c        f(c)     Rel
1  0.50000  1.50000  1.00000  0.54030  0.204217
2  1.00000  1.50000  1.25000  0.01915  0.005271
3  1.25000  1.50000  1.37500  -0.38875  0.094202
4  1.25000  1.37500  1.31250  -0.17256  0.044466
5  1.25000  1.31250  1.28125  -0.07363  0.019598
6  1.25000  1.28125  1.26562  -0.02647  0.007163
7  1.25000  1.26562  1.25781  -0.00347  0.000946
8  1.25000  1.25781  1.25391  0.00789  0.002162
9  1.25391  1.25781  1.25586  0.00222  0.000608
10  1.25586  1.25781  1.25684  -0.00062  0.000169
11  1.25586  1.25684  1.25635  0.00080  0.000219
12  1.25635  1.25684  1.25659  0.00009  0.000025
13  1.25659  1.25684  1.25671  -0.00026  0.000072
14  1.25659  1.25671  1.25665  -0.00009  0.000023
15  1.25659  1.25665  1.25662  0.00000  0.000001

Final Root Approximation = 1.256622
```



Code — A5Q2.m

```
clc;
close all;
clear all;

args = [pi/4, pi, -pi];
for i = 1:3
    arg = args(i);
    fprintf("For initial value x0 = %f\n", arg);
    fprintf("%-6s %-10s %-10s %-10s %-10s %-10s\n", 'iter', 'Pn-1', 'df(Pn-1)', 'Pn', 'f(Pn)',
'err')
    the_func(arg)
    fprintf("\n\n")
end
%arg = 0.1
%the_func(arg)

function the_func(arg)
    for i=1:1000
        x_old = arg;

        df_val = df(x_old);

        x_new = x_old - f(x_old)/df_val;
        f_new = f(x_new);

        rel = abs((x_new - x_old) / (x_new + 1e-12));
        err = abs(x_new - x_old);

        fprintf("%-6d %-10.5f %-10.5f %-10.5f %-10.5f %-10.5f\n", i, x_old, df_val, x_new, f_new,
rel)

        if err < 10^-5
            break;
        end

        arg = x_new;
    end
    fprintf("Ans %f ", x_new)
end

function y = f(x)
    y = 1/2 + 1/4*x^2 - x*sin(x) - 1/2*cos(2*x);
end

function y_prime = df(x)
    y_prime = x/2 - sin(x) - x*cos(x) + sin(2*x);
end



function y_ = f_(x)
    f = x * cos(x) - 2*x^2 + 3*x - 1;
end

function y_fdashprime = df_(x)
    y_dashprime = -1 * x * sin(x) + cos(x) -4*x + 3;
end
```

Output

```
For initial value x0 = 0.785398
iter   Pn-1       df(Pn-1)   Pn         f(Pn)      err
1      0.78540    0.13023    0.02635    0.00017    28.80544
2      0.02635    0.01316    0.01317    0.00004    1.00093
3      0.01317    0.00658    0.00658    0.00001    1.00023
4      0.00658    0.00329    0.00329    0.00000    1.00006
5      0.00329    0.00165    0.00165    0.00000    1.00001
6      0.00165    0.00082    0.00082    0.00000    1.00000
7      0.00082    0.00041    0.00041    0.00000    1.00000
8      0.00041    0.00021    0.00021    0.00000    1.00000
9      0.00021    0.00010    0.00010    0.00000    1.00000
10     0.00010    0.00005    0.00005    0.00000    1.00000
11     0.00005    0.00003    0.00003    0.00000    1.00000
12     0.00003    0.00001    0.00001    0.00000    1.00000
13     0.00001    0.00001    0.00001    0.00000    1.00000
Ans 0.000006

For initial value x0 = 3.141593
iter   Pn-1       df(Pn-1)   Pn         f(Pn)      err
1      3.14159    4.71239    2.61799    0.65448    0.20000
2      2.61799    2.21022    2.32188    0.18489    0.12753
3      2.32188    1.01687    2.14006    0.05186    0.08496
4      2.14006    0.47322    2.03047    0.01417    0.05397
5      2.03047    0.22465    1.96740    0.00376    0.03206
6      1.96740    0.10869    1.93280    0.00097    0.01790
7      1.93280    0.05332    1.91453    0.00025    0.00954
8      1.91453    0.02639    1.90511    0.00006    0.00494
9      1.90511    0.01312    1.90033    0.00002    0.00252
10     1.90033    0.00654    1.89792    0.00000    0.00127
11     1.89792    0.00327    1.89671    0.00000    0.00064
12     1.89671    0.00163    1.89610    0.00000    0.00032
13     1.89610    0.00082    1.89580    0.00000    0.00016
14     1.89580    0.00041    1.89565    0.00000    0.00008
15     1.89565    0.00020    1.89557    0.00000    0.00004
16     1.89557    0.00010    1.89553    0.00000    0.00002
17     1.89553    0.00005    1.89551    0.00000    0.00001
18     1.89551    0.00003    1.89550    0.00000    0.00001
Ans 1.895504

For initial value x0 = -3.141593
iter   Pn-1       df(Pn-1)   Pn         f(Pn)      err
1      -3.14159   -4.71239   -2.61799   0.65448    0.20000
2      -2.61799   -2.21022   -2.32188   0.18489    0.12753
3      -2.32188   -1.01687   -2.14006   0.05186    0.08496
4      -2.14006   -0.47322   -2.03047   0.01417    0.05397
5      -2.03047   -0.22465   -1.96740   0.00376    0.03206
6      -1.96740   -0.10869   -1.93280   0.00097    0.01790
7      -1.93280   -0.05332   -1.91453   0.00025    0.00954
8      -1.91453   -0.02639   -1.90511   0.00006    0.00494
9      -1.90511   -0.01312   -1.90033   0.00002    0.00252
10     -1.90033   -0.00654   -1.89792   0.00000    0.00127
11     -1.89792   -0.00327   -1.89671   0.00000    0.00064
12     -1.89671   -0.00163   -1.89610   0.00000    0.00032
13     -1.89610   -0.00082   -1.89580   0.00000    0.00016
14     -1.89580   -0.00041   -1.89565   0.00000    0.00008
15     -1.89565   -0.00020   -1.89557   0.00000    0.00004
16     -1.89557   -0.00010   -1.89553   0.00000    0.00002
17     -1.89553   -0.00005   -1.89551   0.00000    0.00001
18     -1.89551   -0.00003   -1.89550   0.00000    0.00001
Ans -1.895504
```

Code — A5Q3.m

```
clc
close all
clear all

a = -2;
b = 2;
tol = 10^(-5);
i = 0;
rel = 1;

f_ = @(x) 54*x.^6 + 45*x.^5 + - 102*x.^4 -69*x.^3 + 35*x.^2 + 16*x -4;
%exact_val = roots(f_)

fplot(f_, [-5, 5])
yline(0, '--r', 'LineWidth', 1.5)

[i,a,b,tol,rel] = MAIN(i,a,b,tol,rel);

function [i,a,b,tol,rel] = MAIN(i,a,b,tol,rel)
    fprintf("i   a                  b               c                    f(c)
rel_err\n")

    while rel > tol
        i = i + 1;

        fa = A_FUNC(a);
        fb = A_FUNC(b);

        t = b - fb * (b - a) / (fb - fa);

        fprintf("%d  %0.5f          %0.5f            %0.5f             %0.5f           %f \n", i,
a, b, t, A_FUNC(t), rel)

        if A_FUNC(t) == 0
           fprintf("Root found %0.5f\n", t)
           break;
        end

        rel = abs(t - b);

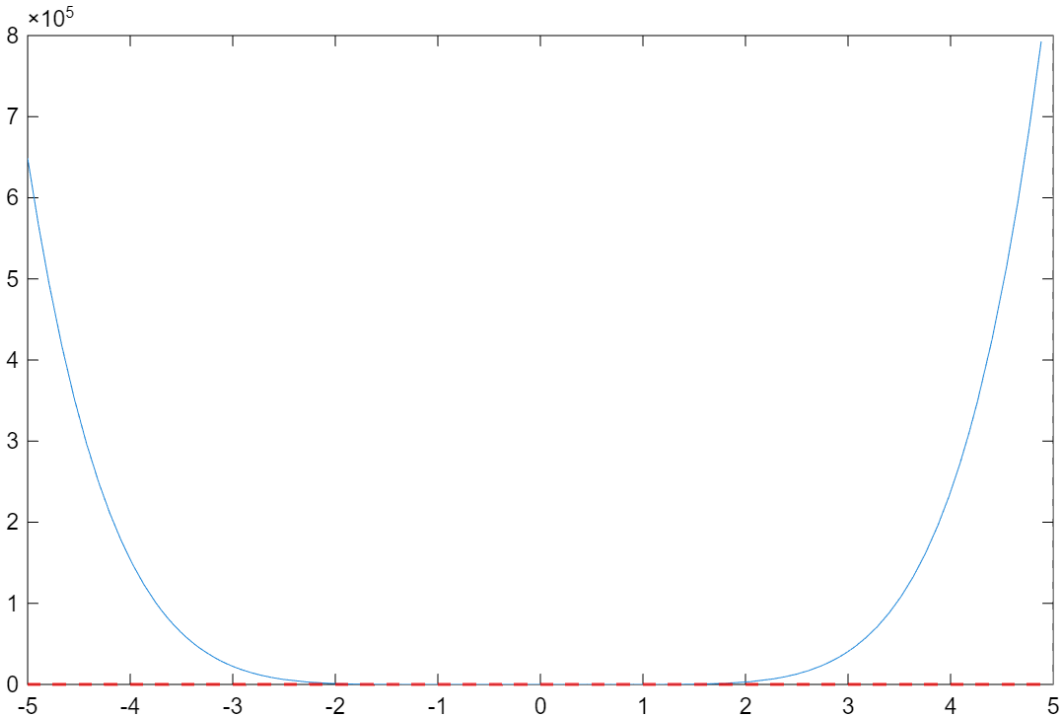
        a = b;
        b = t;
    end
    fprintf("\n Ans = %f\n", t)
end

function f = A_FUNC(x)
    f = 54*x^6 + 45*x^5 + - 102*x^4 -69*x^3 + 35*x^2 + 16*x -4;
end
```

Output

```
i    a                b                c                f(c)                rel_err
1   -2.00000          2.00000          -4.26087          232217.52637          1.000000
2    2.00000         -4.26087           2.07862          3758.42598           6.260870
3   -4.26087          2.07862           2.18292          5237.12832           6.339493
4    2.07862         2.18292          1.81354          1429.15258           0.104292
5    2.18292         1.81354          1.67492          779.41439           0.369372
6    1.81354         1.67492          1.50862          322.79733           0.138627
7    1.67492         1.50862          1.39106          143.97837           0.166295
8    1.50862         1.39106          1.29641          58.25262           0.117559
9    1.39106         1.29641          1.23209          21.40913           0.094654
10   1.29641          1.23209          1.19471          6.16148           0.064320
11   1.23209          1.19471          1.17961          1.09066           0.037375
12   1.19471          1.17961          1.17636          0.07561           0.015103
13   1.17961          1.17636          1.17612          0.00104           0.003248
14   1.17636          1.17612          1.17612          0.00000           0.000242

Ans = 1.176116
```



Code — A5Q4.m

```
clc
close all
clear all

a = 1.3;
b = 2.0;
tol = 10^(-5);
i = 0;
rel = 1;

[i,a,b,tol,rel] = MAIN(i,a,b,tol,rel);

function [i,a,b,tol,rel] = MAIN(i,a,b,tol,rel)
    fprintf("i   a                  b               c                    f(c)
Rel\n")

    c_old = a;

    while rel > tol
        i = i + 1;

        fa = A_FUNC(a);
        fb = A_FUNC(b);

        t = b - (fb * (a - b)) / (fa - fb);

        ft = A_FUNC(t);

        if ft == 0
           fprintf("Exact root found: %0.5f\n", t)
           break;
        end

        rel = abs(t - c_old);
        c_old = t;

        fprintf("%d  %0.5f          %0.5f            %0.5f             %0.5f           %f \n", i,
a, b, t, ft, rel)

        if fa * ft < 0
            b = t;
        else
            a = t;
        end
    end
    fprintf("\nAns = %f\n", c_old)
end

function f = A_FUNC(x)
    f = log(x - 1) + cos(x - 1);
end
```

Output

```
i    a                 b                 c                 f(c)                  Rel
1   1.30000          2.00000           1.52061           0.21476           0.220607
2   1.30000          1.52061           1.41837           0.04236           0.102239
3   1.30000          1.41837           1.40114           0.00717           0.017230
4   1.30000          1.40114           1.39830           0.00118           0.002833
5   1.30000          1.39830           1.39784           0.00019           0.000465
6   1.30000          1.39784           1.39776           0.00003           0.000076
7   1.30000          1.39776           1.39775           0.00001           0.000012
8   1.30000          1.39775           1.39775           0.00000           0.000002

Ans = 1.397749
```

Code — A5Q5.m

```
clc;
close all;
clear all;

%% Step 1: Load Data
A = readmatrix("a.txt");
b = readmatrix("b.txt");

%====================== Part I: Gauss-Jordan Exact Solution ======================
Aug = [A, b];
R = rref(Aug);
x_exact = R(:, end);

disp('==================================================');
disp('--- (i) Exact Solution (Gauss-Jordan) ---');
disp(x_exact);

%====================== Part II: Positive Definiteness Check ======================
disp('--- (ii) Positive Definiteness Check ---');
check_eig(A);

%====================== Part III: Row Reordering for Convergence ======================
% Reorder rows so largest absolute elements sit on the main diagonal
[~, max_cols] = max(abs(A), [], 2);
A_dom = zeros(size(A));
b_dom = zeros(size(b));

for i = 1:length(b)
    A_dom(max_cols(i), :) = A(i, :);
    b_dom(max_cols(i)) = b(i);
end

A_dom;
b_dom;
```

```
% Set parameters for iterative methods
tol = 5e-5; % 5 significant digits (0.5 * 10^(1-5))
max_iter = 500;
x0 = zeros(size(b));

%====================== Part IV: Iterative Solvers ======================
disp('=================================================');
disp('--- Iterative Results (Correct to 5 Significant Digits) ---');

% (a) Jacobi Method
[x_jacobi, iter_j, status_j] = jacobi_method(A_dom, b_dom, x0, tol, max_iter);
fprintf('Jacobi Solution (%d iterations, Status: %s):\n', iter_j, status_j);
disp(x_jacobi);

% (b) Gauss-Seidel Method
[x_gs, iter_gs, status_gs] = gauss_seidel_method(A_dom, b_dom, x0, tol, max_iter);
fprintf('Gauss-Seidel Solution (%d iterations, Status: %s):\n', iter_gs, status_gs);
disp(x_gs);

% (c) SOR Method (omega = 1.1)
omega = 1.1;
[x_sor, iter_sor, status_sor] = sor_method(A_dom, b_dom, x0, omega, tol, max_iter);
fprintf('SOR (omega = 1.1) Solution (%d iterations, Status: %s):\n', iter_sor, status_sor);
disp(x_sor);

% (d) Additional Table Output via sor_ (omega = 1.2)
disp('--- Custom SOR Table Output (omega = 1.2) ---');
x_init = zeros(size(b));
[x_sor_tbl, iter_tbl, err_tbl, table_data] = sor_(A_dom, b_dom, x_init, 1.2, 700);
T = array2table(table_data, 'VariableNames', {'iteration','x1','x2','x3','x4','x5', 'error'});
disp(T);
disp("The solution:");
disp(x_sor_tbl);



%==========================================================================
%                          LOCAL FUNCTIONS
%==========================================================================

function check_eig(a)
    % Positive definiteness requires symmetry: A = A'
    if ~isequal(a, a')
        disp("Matrix A is NOT Positive Definite.");
        disp("Reason: Matrix is asymmetric (A ~= A').");
        return;
    end

    eigen = eig(a);
    if all(eigen > 0)
        disp("Matrix A is Positive Definite.");
    else
        disp("Matrix A is NOT Positive Definite (Eigenvalues <= 0).");
    end
end

function [x, iter, status] = jacobi_method(A, b, x0, tol, max_iter)
    n = length(b);
    D = diag(diag(A));
    LU = A - D;
    x = x0;
```

```
    status = 'Converged';

    for iter = 1:max_iter
        x_new = D \ (b - LU * x);

        % Divergence Guard
        if any(isnan(x_new)) || any(isinf(x_new)) || max(abs(x_new)) > 1e10
            status = 'Diverged';
            x = x_new;
            return;
        end

        % Relative Error Check
        if max(abs(x_new - x) ./ (abs(x_new) + 1e-12)) < tol
            x = x_new;
            return;
        end
        x = x_new;
    end
    status = 'Max iterations reached';
end

function [x, iter, status] = gauss_seidel_method(A, b, x0, tol, max_iter)
    L_D = tril(A); % Lower triangular + Diagonal
    U = triu(A, 1); % Upper triangular
    x = x0;
    status = 'Converged';

    for iter = 1:max_iter
        x_new = L_D \ (b - U * x);

        if any(isnan(x_new)) || any(isinf(x_new)) || max(abs(x_new)) > 1e10
            status = 'Diverged';
            x = x_new;
            return;
        end

        if max(abs(x_new - x) ./ (abs(x_new) + 1e-12)) < tol
            x = x_new;
            return;
        end
        x = x_new;
    end
    status = 'Max iterations reached';
end

function [x, iter, status] = sor_method(A, b, x0, omega, tol, max_iter)
    D = diag(diag(A));
    L = tril(A, -1);
    U = triu(A, 1);

    M = D + omega * L;
    N = (1 - omega) * D - omega * U;
    c = omega * (M \ b);
    T = M \ N;

    x = x0;
    status = 'Converged';
```

```
    for iter = 1:max_iter
        x_new = T * x + c;

        if any(isnan(x_new)) || any(isinf(x_new)) || max(abs(x_new)) > 1e10
            status = 'Diverged';
            x = x_new;
            return;
        end

        if max(abs(x_new - x) ./ (abs(x_new) + 1e-12)) < tol
            x = x_new;
            return;
        end
        x = x_new;
    end
    status = 'Max iterations reached';
end

function [iter, x, err, table_data] = gauss_sed(A, b, x0, max_iter)
    n = length(b);
    tol = 1e-5;
    x = x0;

    table_data = zeros(max_iter + 1, n + 2);
    table_data(1,:) = [0, x', 0];

    for iter = 1:max_iter
        x_old = x;

        for i = 1:n
            d = b(i);
            for j = 1:n
                if i ~= j
                    d = d - A(i,j) * x(j);
                end
            end
            x(i) = d / A(i,i);
        end

        err = max(abs(x - x_old));
        table_data(iter+1,:) = [iter, x', err];

        if err < tol
            break;
        end
    end
    table_data = table_data(1:iter+1,:);
end

function [x, iter, err, table_data] = sor_(A, b, x0, w, max_iter)
    n = length(b);
    tol = 1e-5;
    x = x0;

    table_data = zeros(max_iter + 1, n + 2);
    table_data(1,:) = [0, x', 0];

    for iter = 1:max_iter
        x_old = x;
```

```
        for i = 1:n
            d = b(i);
            for j = 1:n
                if i ~= j
                    d = d - A(i,j) * x(j);
                end
            end
            x_gs = d / A(i,i);
            x(i) = (1 - w) * x_old(i) + w * x_gs;
        end

        err = max(abs(x - x_old));
        table_data(iter+1,:) = [iter, x', err];

        if err < tol
            break;
        end
    end
    table_data = table_data(1:iter+1,:);
end
```

Input

a.txt:

```
4 1 1 0 1
-1 -3 1 1 0
-1 -1 -1 4 0
2 1 5 -1 -1
0 2 -1 1 4
```

b.txt:

```
6
6
6
6
6
```

Output

```
==================================================
--- (i) Exact Solution (Gauss-Jordan) ---
    0.7866
   -1.0026
    1.8663
    1.9126
    1.9897

--- (ii) Positive Definiteness Check ---
Matrix A is NOT Positive Definite.
Reason: Matrix is asymmetric (A ~= A').
==================================================
--- Iterative Results (Correct to 5 Significant Digits) ---
Jacobi Solution (16 iterations, Status: Converged):
    0.7866
   -1.0026
    1.8663
    1.9126
    1.9897

Gauss-Seidel Solution (9 iterations, Status: Converged):
```

```
    0.7866
   -1.0026
    1.8663
    1.9126
    1.9897

SOR (omega = 1.1) Solution (10 iterations, Status: Converged):
    0.7866
   -1.0026
    1.8663
    1.9126
    1.9897

--- Custom SOR Table Output (omega = 1.2) ---
    iteration      x1          x2          x3          x4          x5          error
    _________    _______    ________    _______    _______    _______    __________

        0              0           0          0          0          0             0
        1            1.8       -3.12     1.3248     1.8014      3.529         3.529
        2        0.91986    -0.89345     2.2272     2.1158     1.6637        2.2266
        3        0.71678     -0.7708     1.7426     1.8834     1.8875       0.48467
        4        0.79887      -1.115     1.8806     1.8927     2.0879       0.34419
        5        0.78417    -0.98134     1.8783     1.9258      1.957       0.13366
        6        0.78698    -0.99687     1.8577     1.9092     1.9913      0.034299
        7        0.78697     -1.0087     1.8689     1.9123     1.9939      0.011794
        8        0.78636     -1.0003     1.8663     1.9132     1.9873     0.0083408
        9        0.78672     -1.0028     1.8659     1.9123     1.9903     0.0029522
       10        0.78663     -1.0028     1.8665     1.9126     1.9898    0.00061439
       11        0.78662     -1.0024     1.8663     1.9126     1.9896    0.00039042
       12        0.78664     -1.0026     1.8663     1.9126     1.9898    0.00019712
       13        0.78663     -1.0026     1.8663     1.9126     1.9897    6.0839e-05
       14        0.78663     -1.0026     1.8663     1.9126     1.9897    1.6144e-05
       15        0.78663     -1.0026     1.8663     1.9126     1.9897    1.1196e-05
       16        0.78663     -1.0026     1.8663     1.9126     1.9897    4.4312e-06

The solution:
    0.7866
   -1.0026
    1.8663
    1.9126
    1.9897
```

Code — A5Q6.m

```
clc;
clear all;
close all;

% Initial interval
a = 2;
b = 3;

% Run root-finding algorithms
[x_b, re_b]     = bisection(a, b);
[x_new, re_ne]  = newton(a, b);
[x_sec, re_sec] = secant(a, b);
[x_fal, re_fal] = false(a, b);

% Pad vectors with NaN to match maximum iteration length
```

```
n_max = max([length(x_b), length(x_new), length(x_sec), length(x_fal)]);

x_b(end+1:n_max)    = NaN;
x_new(end+1:n_max)  = NaN;
x_sec(end+1:n_max)  = NaN;
x_fal(end+1:n_max)  = NaN;

re_b(end+1:n_max)   = NaN;
re_ne(end+1:n_max)  = NaN;
re_sec(end+1:n_max) = NaN;
re_fal(end+1:n_max) = NaN;

% 1. Run algorithms
[x_b, re_b]     = bisection(a, b);
[x_new, re_ne]  = newton(a, b);
[x_sec, re_sec] = secant(a, b);
[x_fal, re_fal] = false(a, b);

% 2. Find maximum length
n_max = max([length(x_b), length(x_new), length(x_sec), length(x_fal)]);

% 3. Pad roots by repeating the final value
x_b(end+1:n_max)   = x_b(end);
x_new(end+1:n_max) = x_new(end);
x_sec(end+1:n_max) = x_sec(end);
x_fal(end+1:n_max) = x_fal(end);

% 4. Pad errors with 0
re_b(end+1:n_max)   = 0;
re_ne(end+1:n_max)  = 0;
re_sec(end+1:n_max) = 0;
re_fal(end+1:n_max) = 0;

% 5. NOW create the matrix (or table) AFTER padding is done!
matrix = [x_b(:), x_new(:), x_sec(:), x_fal(:), re_b(:), re_ne(:), re_sec(:), re_fal(:)];

% Optional: Display it cleanly as a Table so you see column names
Iteration = (1:n_max)';
T = table(Iteration, x_b(:), x_new(:), x_sec(:), x_fal(:), re_b(:), re_ne(:), re_sec(:),
re_fal(:));
T.Properties.VariableNames = {'Iteration', 'x_bisection', 'x_newton', 'x_secant', 'x_false',
'err_bisection', 'err_newton', 'err_secant', 'err_false'};

disp(' ');
disp('==========================================================================================
==================');
disp('                                    ROOT-FINDING METHODS ITERATION TABLE');
disp('==========================================================================================
==================');
disp(T);

% Plot Relative Error vs Iteration
figure;
plot(1:n_max, re_b, 'r-o', 'LineWidth', 1.5, 'DisplayName', 'Bisection');
%Font adjustment
ax = gca;
ax.FontSize = 14;

hold on;
```

```
plot(1:n_max, re_ne, 'g-s', 'LineWidth', 1, 'DisplayName', 'Newton-Raphson');
plot(1:n_max, re_sec, 'b-d', 'LineWidth', 0.5, 'DisplayName', 'Secant');
plot(1:n_max, re_fal, 'm-^', 'LineWidth', 0.25, 'DisplayName', 'False Position');
hold off;
%plot function to do

xlabel('Iteration');
ylabel('Error');
title('Convergence Comparison of Root-Finding Methods');
legend('Location', 'northeast');
grid on;



%% ========================================================================
%                              LOCAL FUNCTIONS
% ========================================================================

function y = f(x)
    y = x^3 - x - exp(x) - 2;
end

function dy = df(x)
    dy = 3*x^2 - 1 - exp(x);
end

function [x_b, re_b] = bisection(a, b)
    i = 1;
    tol = 1e-5;
    FA = f(a);
    po = 0;
    x_b = [];
    re_b = [];

    if (f(a) * f(b)) < 0
        while i <= 100
            p = a + (b - a)/2;
            FP = f(p);
            re_b(i) = abs((p - po)/p) * 100;
            x_b(i) = p;

            if FP == 0 || abs((b - a)/2) < tol
                break;
            end

            i = i + 1;
            if (FA * FP) > 0
                a = p;
            else
                b = p;
            end
            po = p;
        end
    elseif (f(a) * f(b)) >= 0
        disp('Bisection not possible: f(a) and f(b) must have opposite signs.');
    end
end

function [x_new, re_ne] = newton(a, b)
    i = 1;
    tol = 1e-5;
```

```
    p0 = (a + b)/2;
    x_new = [];
    re_ne = [];

    if df(p0) == 0
        return;
    else
        while i <= 100
            p = p0 - f(p0)/df(p0);
            re_ne(i) = abs((p - p0)/p) * 100;
            x_new(i) = p;

            if abs(p0 - p) < tol
                break;
            end

            i = i + 1;
            p0 = p;
        end
    end
end

function [x_sec, re_sec] = secant(a, b)
    i = 1;
    tol = 1e-5;
    x_sec = [];
    re_sec = [];

    if df(a) < df(b)
        p1 = a;
        p0 = b;
    else
        p1 = b;
        p0 = a;
    end

    q0 = f(p0);
    q1 = f(p1);

    while i <= 100
        q0 = f(p0);
        q1 = f(p1);
        if (q1 - q0) == 0
            break;
        end

        p = p1 - q1 * (p1 - p0) / (q1 - q0);
        re_sec(i) = abs((p - p1)/p) * 100;
        x_sec(i) = p;

        if abs(p - p1) < tol
            %fprintf('Secant Output: %f\n', p);
            break;
        end

        i = i + 1;
        p0 = p1;
        p1 = p;
    end
end
```

```
function [x_fal, re_fal] = false(a, b)
    tol = 1e-5;
    i = 1;
    p0 = a;
    p1 = b;
    q0 = f(p0);
    q1 = f(p1);
    x_fal = [];
    re_fal = [];

    while i <= 100
        p = p1 - q1 * (p1 - p0) / (q1 - q0);
        re_fal(i) = abs((p - p1)/p) * 100;
        x_fal(i) = p;

        if abs(p - p1) < tol
            break;
        end

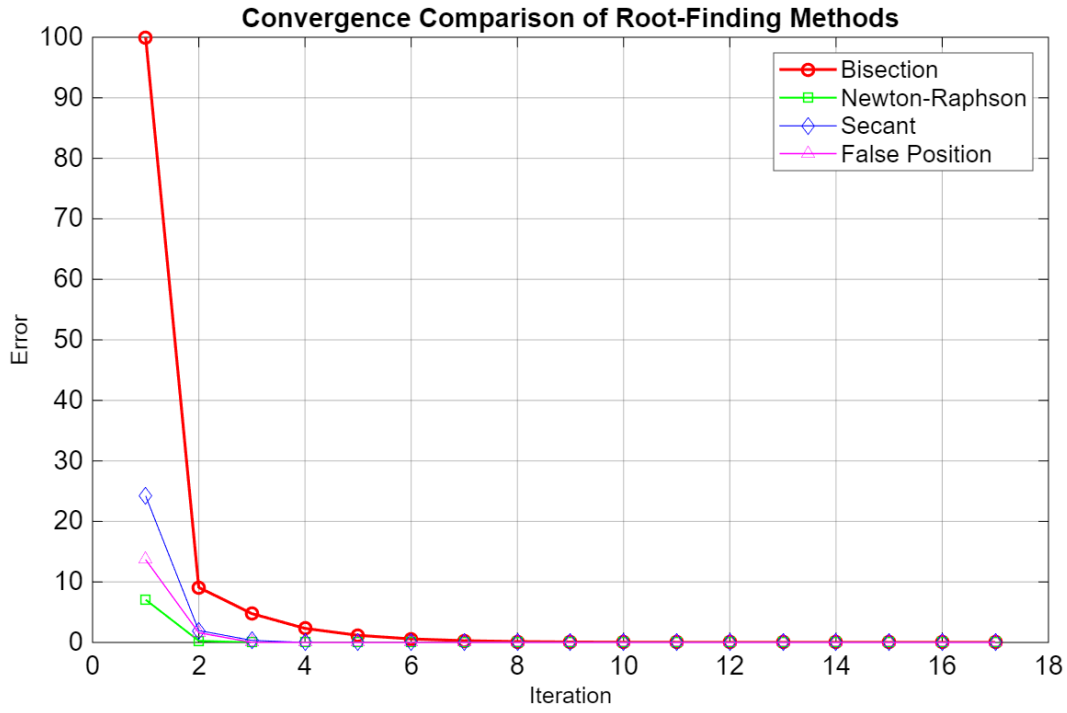
        i = i + 1;
        q = f(p);

        if q * q1 < 0
            p0 = p1;
            q0 = q1;
        end
        p1 = p;
        q1 = q;
    end
end
```

Output

```
=================================================================================================================
                                    ROOT-FINDING METHODS ITERATION TABLE
=================================================================================================================
    Iteration    x_bisection    x_newton    x_secant    x_false    err_bisection    err_newton    err_secant    err_false
    _________    ___________    ________    ________    _______    _____________    __________    __________    _________

        1             2.5         2.6899       2.639      2.639             100          7.0611        24.214        13.679
        2            2.75         2.6827      2.6919     2.6821          9.0909         0.26867        1.9662        1.6051
        3           2.625         2.6827      2.6827     2.6827          4.7619      0.00023203       0.34575      0.024171
        4          2.6875         2.6827      2.6827     2.6827          2.3256               0     0.0019821    0.00027362
        5          2.6562         2.6827      2.6827     2.6827          1.1765               0    2.2136e-06             0
        6          2.6719         2.6827      2.6827     2.6827          0.5848               0             0             0
        7          2.6797         2.6827      2.6827     2.6827         0.29155               0             0             0
        8          2.6836         2.6827      2.6827     2.6827         0.14556               0             0             0
        9          2.6816         2.6827      2.6827     2.6827        0.072833               0             0             0
       10          2.6826         2.6827      2.6827     2.6827        0.036403               0             0             0
       11          2.6831         2.6827      2.6827     2.6827        0.018198               0             0             0
       12          2.6829         2.6827      2.6827     2.6827          0.0091               0             0             0
       13          2.6827         2.6827      2.6827     2.6827       0.0045502               0             0             0
       14          2.6827         2.6827      2.6827     2.6827       0.0022752               0             0             0
       15          2.6827         2.6827      2.6827     2.6827       0.0011376               0             0             0
       16          2.6827         2.6827      2.6827     2.6827      0.00056878               0             0             0
       17          2.6827         2.6827      2.6827     2.6827      0.00028439               0             0             0
```



Code — rocket_nozzle.m

```
generate_rao_bell_nozzle()

function generate_rao_bell_nozzle()
    % --- 1. Nozzle Input Parameters ---
    R_t = 15.0;             % Throat radius (mm)
    expansion_ratio = 8.0; % Area expansion ratio (A_e / A*)
    percent_length = 85;    % % length of equivalent 15-deg conical nozzle
    theta_i = 28;           % Initial divergent parabola wall angle (degrees)
    theta_e = 9;            % Exit wall angle (degrees)
    num_points = 100;       % Resolution for divergent bell section

    % --- 2. Geometric Calculations ---
    R_e = R_t * sqrt(expansion_ratio);

    % Reference 15-degree conical length
    L_15 = (R_e - R_t) / tand(15);
    L_n = (percent_length / 100.0) * L_15;

    % Radians for trigonometric evaluations
    t_i = deg2rad(theta_i);
    t_e = deg2rad(theta_e);

    % --- 3. Convergent Region ---
    R_chamber = R_t * 2.5;
    theta_conv = deg2rad(30); % Standard 30-deg contraction angle
```

```
    x_conv_start = - ((R_chamber - R_t) / tan(theta_conv));
    x_conv = linspace(x_conv_start, -0.382 * R_t, round(num_points / 2));
    y_conv = R_chamber - (x_conv - x_conv_start) * tan(theta_conv);

    % --- 4. Throat Region (Circular Arc Downstream) ---
    r_dn = 0.382 * R_t;
    x_i = r_dn * sin(t_i);
    y_i = R_t + r_dn * (1 - cos(t_i));

    theta_arc = linspace(0, t_i, 20);
    x_throat_arc = r_dn * sin(theta_arc);
    y_throat_arc = R_t + r_dn * (1 - cos(theta_arc));

    % --- 5. Divergent Parabolic Bell Section ---
    x_e = L_n;
    y_e = R_e;

    m1 = tan(t_i);
    m2 = tan(t_e);

    % Quadratic Bezier Control Point Intersection
    x_c = (y_e - y_i + m1 * x_i - m2 * x_e) / (m1 - m2);
    y_c = y_i + m1 * (x_c - x_i);

    % Parabolic parameter t from 0 to 1
    t = linspace(0, 1, num_points);
    x_div = (1 - t).^2 * x_i + 2 * (1 - t) .* t * x_c + t.^2 * x_e;
    y_div = (1 - t).^2 * y_i + 2 * (1 - t) .* t * y_c + t.^2 * y_e;

    % Combine full profile coordinates
    x_full = [x_conv, x_throat_arc, x_div];
    y_full = [y_conv, y_throat_arc, y_div];

    % --- 6. Visualization in MATLAB ---
    fig = figure('Color', [0.12 0.12 0.18], 'Position', [100 100 1000 500]);
    ax = axes('Parent', fig, 'Color', [0.12 0.12 0.18], 'XColor', 'w', 'YColor', 'w');
    hold(ax, 'on');
    grid(ax, 'on');
    set(ax, 'GridColor', [0.3 0.3 0.4], 'GridAlpha', 0.6, 'GridLineStyle', '--');

    % Upper & Lower Nozzle Walls
    plot(ax, x_full, y_full, 'Color', '#00d2ff', 'LineWidth', 2.5, 'DisplayName', 'Nozzle Wall
Contour');
    plot(ax, x_full, -y_full, 'Color', '#00d2ff', 'LineWidth', 2.5, 'HandleVisibility', 'off');

    % Centerline Axis
    yline(ax, 0, '--w', 'Centerline Axis', 'Alpha', 0.5, 'LineWidth', 1, 'Color', 'w',
'DisplayName', 'Centerline Axis');

    % Gas Flow Domain Fill
    patch(ax, [x_full, fliplr(x_full)], [y_full, -fliplr(y_full)], [0 0.82 1], ...
          'FaceAlpha', 0.08, 'EdgeColor', 'none', 'HandleVisibility', 'off');

    % Key Stations
    xline(ax, x_conv_start, ':r', 'Chamber Inlet', 'Color', '#ff4757', 'Alpha', 0.7, 'LineWidth',
1.2);
    xline(ax, 0, ':y', 'Throat (M = 1.0)', 'Color', '#ffa502', 'Alpha', 0.7, 'LineWidth', 1.2);
    xline(ax, L_n, ':g', 'Exit Geometry', 'Color', '#2ed573', 'Alpha', 0.7, 'LineWidth', 1.2);
```

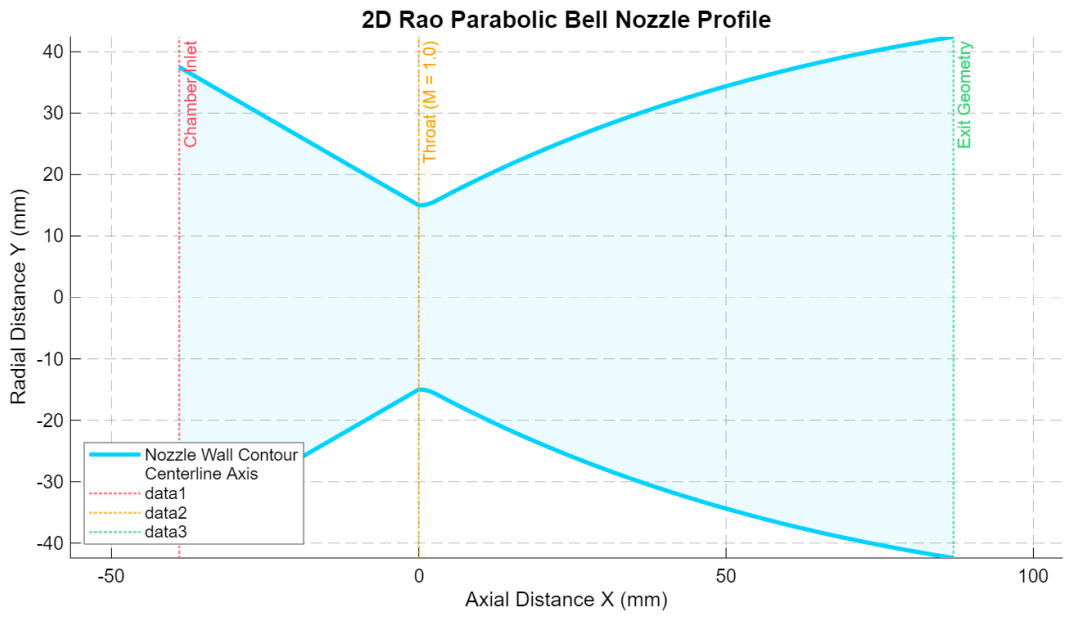
```
    % Formatting
    title(ax, '2D Rao Parabolic Bell Nozzle Profile', 'Color', 'w', 'FontSize', 13, 'FontWeight',
'bold');
    xlabel(ax, 'Axial Distance X (mm)', 'Color', 'w', 'FontSize', 11);
    ylabel(ax, 'Radial Distance Y (mm)', 'Color', 'w', 'FontSize', 11);
    axis(ax, 'equal');
    legend(ax, 'Location', 'southwest', 'TextColor', 'w', 'Color', [0.1 0.1 0.15], 'EdgeColor',
[0.3 0.3 0.4]);

    % --- 7. Export CSV for SolidWorks (X, Y, Z) ---
    z_full = zeros(size(x_full));
    data_out = [x_full', y_full', z_full'];

    writematrix(data_out, 'solidworks_nozzle_curve.csv');
    fprintf('Exported %d points to "solidworks_nozzle_curve.csv"\n', length(x_full));
end
```

Output

```
Exported 170 points to "solidworks_nozzle_curve.csv"
```



Code — sor.m

```
clc; close all; clear all;

A = [4 1 2;
     3 5 1;
     1 1 3];

b = [4 7 3]';

aug = [A, b];

x0 = [0; 0; 0;];
```

```
[x, iter, err, table_data] = sor_(A, b, x0, 1.2, 700);

T = array2table(table_data, 'VariableNames', {'iteration','x1','x2','x3','error'});
disp(T);
disp("The solution ")
disp(x)

function [x, iter, err, table_data] = sor_(A, b, x0, w, max_iter)
    n = length(b);
    tol = 1e-5;
    x = x0;

    table_data = zeros(max_iter + 1, n + 2);
    table_data(1,:) = [0, x', 0];

    for iter = 1:max_iter
        x_old = x;
        for i = 1:n
            d = b(i);
            x_old = x;

            for j = 1:n
                if i ~= j
                    d = d - A(i,j)*x(j);
                end
            end

            x_gs = d/A(i,i);
            x(i) = (1-w) * x_old(i) + w*x_gs;

        end

        err = max(abs(x-x_old));
        table_data(iter+1,:) = [iter,x',err];

        if err < tol
            break;
        end

    end
    table_data = table_data(1:iter+1,:);
end
```

Output

iteration	x1	x2	x3	error
0	0	0	0	0
1	1.2	0.816	0.3936	0.3936
2	0.47904	1.0774	0.49869	0.10509
3	0.48175	0.99797	0.50837	0.0096812
4	0.49923	0.99895	0.49905	0.009322
5	0.50104	0.99969	0.4999	0.00084579
6	0.49995	1.0001	0.49999	9.3764e-05
7	0.49998	0.99999	0.50001	2.1394e-05
8	0.5	1	0.5	1.5091e-05
9	0.5	1	0.5	1.7604e-06

```
The solution
    0.5000
    1.0000
    0.5000
```

Assignment 6

Code — A6Q1.m

```
clc; clear; close all;

f = @(t,y) (2 - 2*t*y) / (t^2 + 1);
y_exact_fn = @(t) (2*t + 1) ./ (t.^2 + 1);

step_sizes = [0.5, 0.25, 0.1, 0.01];
colors = ["#D95319", "#EDB120", "#7E2F8E", "#77AC30"];

t0 = 0; target_x = 1; y0 = 1;

plot_euler_results(target_x, t0, y0, step_sizes, colors, f, y_exact_fn);



%%=========================================================================%%
%%TODO: have to understand and implement the plot function from scratch%%
%%=========================================================================%%

dis_result(target_x, t0, y0, step_sizes, f, y_exact_fn);



function  dis_result(target_x, t0, y0, step_sizes, f, y_exact_fn)
    for i = 1:length(step_sizes)
        h = step_sizes(i);

        [iter, x_history, y_history, final_err] = euler(target_x, t0, y0, h, f, y_exact_fn);

        fprintf('\n=== TABLE FOR h = %.2f ===\n', h);
        fprintf('%-10s %-12s %-12s %-12s\n', 't', 'Exact', 'Euler Appr');
        fprintf('%s\n', repmat('-', 1, 48));

        for k = 1:length(x_history)
            t_val = x_history(k);
            y_appr = y_history(k);
            y_exact = y_exact_fn(t_val);
            %err_val = abs(y_exact - y_appr);

            fprintf('%-10.2f %-12.6f %-12.6f \n', t_val, y_exact, y_appr);
        end
        fprintf('\n');
    end
end

function plot_euler_results(target_x, t0, y0, step_sizes, colors, f, y_exact_fn)



    t_common = (t0:0.1:target_x)';
    sol_matrix = zeros(length(t_common), length(step_sizes));

    figure('Position', [100, 100, 800, 500]);
    hold on; grid on;

    t_fine = 0:0.001:target_x;
    plot(t_fine, y_exact_fn(t_fine), 'k-', 'LineWidth', 2.5, 'DisplayName', 'Exact Solution');
```

```
    for i = 1:length(step_sizes)
        h = step_sizes(i);

        [iter, x_history, y_history, final_err] = euler(target_x, t0, y0, h, f, y_exact_fn);

        sol_matrix(:, i) = interp1(x_history, y_history, t_common, 'linear');

        plot(x_history, y_history, 'o--', 'LineWidth', 1.5, 'MarkerSize', 5, ...
            'Color', colors(i), ...
            'DisplayName', sprintf('h = %.2f (Iter: %d, Err: %.4f)', h, iter, final_err));
    end

    xlabel('t', 'FontSize', 12);
    ylabel('y(t)', 'FontSize', 12);
    title("Euler's Method Approximation vs. Exact Solution", 'FontSize', 14);
    legend('Location', 'southeast', 'FontSize', 10);
    hold off;

%    fprintf('\n%-10s %-12s %-12s %-12s %-12s %-12s\n', 't', 'Exact', 'h=0.5', 'h=0.25', 'h=0.1',
'h=0.01');
%    fprintf('%s\n', repmat('-', 1, 72));

%    for k = 1:length(t_common)
%        t_val = t_common(k);
%        y_exact = y_exact_fn(t_val);
%        fprintf('%-10.2f %-12.6f %-12.6f %-12.6f %-12.6f %-12.6f\n', ...
%            t_val, y_exact, sol_matrix(k, 1), sol_matrix(k, 2), sol_matrix(k, 3), sol_matrix(k,
4));
%    end
%    fprintf('\n');
end



function [iter, x_history, y_history, err] = euler(x, x0, y, h, f, y_exact_fn)

    x_new = x0;
    iter = 0;

    x_history = x_new;
    y_history = y;

    while (x - x_new) > 1e-9
        %step = min(h, x - x_new);

        y = y + h * f(x_new, y);
        x_new = x_new + h;
        iter = iter + 1;

        x_history(end+1) = x_new; %#ok<AGROW>
        y_history(end+1) = y;     %#ok<AGROW>
    end

    exact_sol = y_exact_fn(x_new);
    err = abs(exact_sol - y);
end
```

Output

```
=== TABLE FOR h = 0.50 ===
t          Exact        Euler Appr   ------------------------------------------------
0.00       1.000000     1.000000
0.50       1.600000     2.000000
1.00       1.500000     2.000000

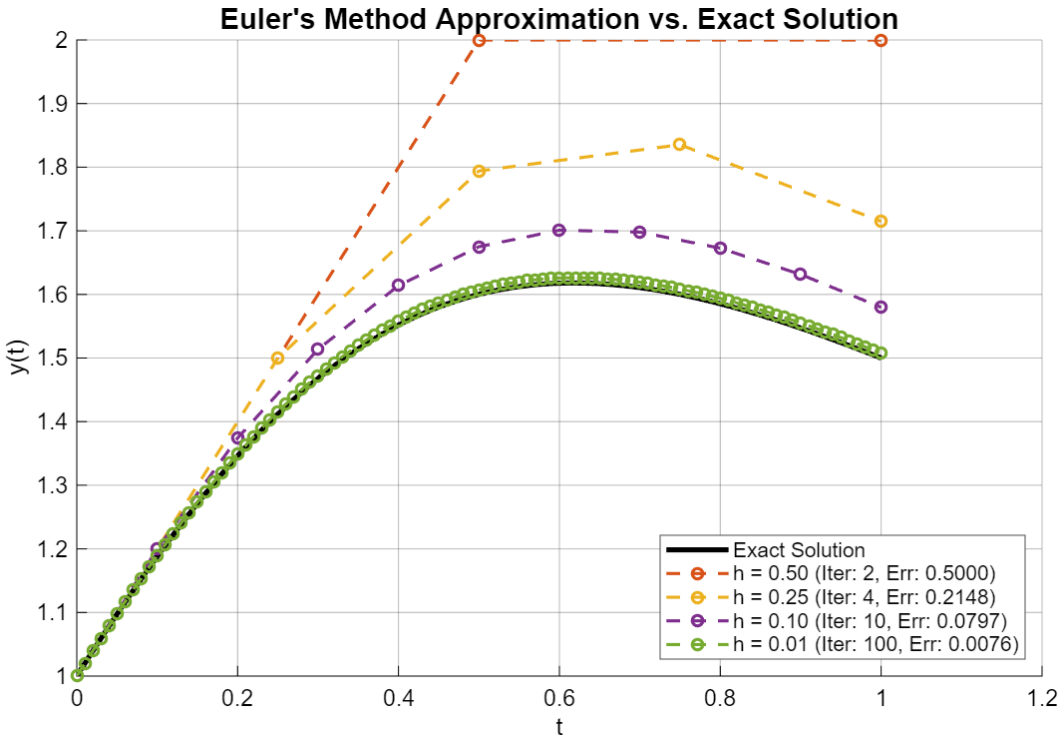

=== TABLE FOR h = 0.25 ===
t          Exact        Euler Appr   ------------------------------------------------
0.00       1.000000     1.000000
0.25       1.411765     1.500000
0.50       1.600000     1.794118
0.75       1.600000     1.835294
1.00       1.500000     1.714824


=== TABLE FOR h = 0.10 ===
t          Exact        Euler Appr   ------------------------------------------------
0.00       1.000000     1.000000
0.10       1.188119     1.200000
0.20       1.346154     1.374257
0.30       1.467890     1.513709
0.40       1.551724     1.613872
0.50       1.600000     1.674984
0.60       1.617647     1.700985
0.70       1.610738     1.697957
0.80       1.585366     1.672646
0.90       1.546961     1.631412
1.00       1.500000     1.579669


=== TABLE FOR h = 0.01 ===
t          Exact        Euler Appr   ------------------------------------------------
0.00       1.000000     1.000000
0.01       1.019898     1.020000
0.02       1.039584     1.039794
0.03       1.059047     1.059370
0.04       1.078275     1.078717
0.05       1.097257     1.097824
0.06       1.115982     1.116679
0.07       1.134441     1.135272
0.08       1.152623     1.153593
0.09       1.170519     1.171631
0.10       1.188119     1.189379
0.11       1.205414     1.206826
0.12       1.222397     1.223963
0.13       1.239060     1.240783
0.14       1.255394     1.257279
0.15       1.271394     1.273441
0.16       1.287051     1.289265
0.17       1.302362     1.304743
0.18       1.317319     1.319870
0.19       1.331918     1.334640
0.20       1.346154     1.349048
0.21       1.360023     1.363090
0.22       1.373522     1.376762
0.23       1.386646     1.390061
0.24       1.399395     1.402983
0.25       1.411765     1.415526
```

0.26	1.423754	1.427688
0.27	1.435362	1.439468
0.28	1.446588	1.450864
0.29	1.457430	1.461876
0.30	1.467890	1.472503
0.31	1.477967	1.482747
0.32	1.487663	1.492606
0.33	1.496979	1.502083
0.34	1.505916	1.511179
0.35	1.514477	1.519895
0.36	1.522663	1.528234
0.37	1.530478	1.536199
0.38	1.537924	1.543791
0.39	1.545005	1.551015
0.40	1.551724	1.557874
0.41	1.558086	1.564372
0.42	1.564094	1.570512
0.43	1.569753	1.576299
0.44	1.575067	1.581737
0.45	1.580042	1.586831
0.46	1.584681	1.591587
0.47	1.588992	1.596009
0.48	1.592978	1.600102
0.49	1.596645	1.603872
0.50	1.600000	1.607325
0.51	1.603047	1.610467
0.52	1.605793	1.613302
0.53	1.608244	1.615838
0.54	1.610406	1.618081
0.55	1.612284	1.620035
0.56	1.613886	1.621709
0.57	1.615216	1.623107
0.58	1.616283	1.624237
0.59	1.617091	1.625104
0.60	1.617647	1.625715
0.61	1.617958	1.626076
0.62	1.618029	1.626194
0.63	1.617868	1.626075
0.64	1.617480	1.625725
0.65	1.616872	1.625151
0.66	1.616049	1.624359
0.67	1.615018	1.623355
0.68	1.613786	1.622145
0.69	1.612357	1.620736
0.70	1.610738	1.619133
0.71	1.608936	1.617342
0.72	1.606955	1.615370
0.73	1.604801	1.613222
0.74	1.602481	1.610904
0.75	1.600000	1.608422
0.76	1.597363	1.605781
0.77	1.594576	1.602987
0.78	1.591644	1.600045
0.79	1.588572	1.596961
0.80	1.585366	1.593740
0.81	1.582030	1.590386
0.82	1.578570	1.586905
0.83	1.574990	1.583303
0.84	1.571295	1.579583
0.85	1.567489	1.575750
0.86	1.563578	1.571809
0.87	1.559565	1.567765

```
0.88        1.555455      1.563622
0.89        1.551253      1.559384
0.90        1.546961      1.555056
0.91        1.542585      1.550641
0.92        1.538128      1.546143
0.93        1.533594      1.541567
0.94        1.528987      1.536917
0.95        1.524310      1.532195
0.96        1.519567      1.527406
0.97        1.514761      1.522552
0.98        1.509896      1.517638
0.99        1.504974      1.512667
1.00        1.500000      1.507642
```



Code — A6Q2.m

```
clc; close all; clear all;

%========Define Functions================
y_anal = @(t) (2*t  + 1) / (t^2 + 1);
y_prime = @(t,y) (2 - 2*t*y) ./ (t^2 + 1);
%=========================================

%================Constants=============
h_vals = [0.5, 0.25, 0.1, 0.01];
t0 = 0; y0 = 1;

disp("[Comparison]")
fprintf("\n")
% Print the overall Table Header
fprintf("%-6s    %-10s    %-10s    %-10s    %-10s\n", 'h', 'Total_Iter', 'Exact_Val',
'Approx_Val', 'Abs_Error');
fprintf("-----------------------------------------------------------------\n");

for i = 1:length(h_vals)
    [iter, t_vals, y_vals, exact, err] = euler(1, t0, y0, h_vals(i), y_prime, y_anal);

    % Use 'end' to grab the final computed value at target_T
    final_y = y_vals(end);

    % Print a clean, formatted row for each step size
    fprintf("%-6.2f    %-10d    %-10.5f    %-10.5f    %-10.5e\n", ...
            h_vals(i), iter, exact, final_y, err);
end



fprintf("\n")
disp("[Detailed Table]")
for i = 1:length(h_vals)

    current_h = h_vals(i); % Changed name to avoid variable collisions
    [total_steps, t_vals, y_vals, final_exact, final_err] = euler(1, t0, y0, current_h, y_prime,
y_anal);

    % Fixed format token from .2%f to %.2f
    fprintf("\nTable for h = %.2f\n", current_h);

    % Cleaned up 5 structural columns matching 5 column names
    fprintf("%-6s  %-10s  %-10s  %-10s  %-10s\n", "iter", "t_vals", "exact", "y_vals", "error");
    fprintf("=================================================================\n");

    for j = 1:length(t_vals)
        % Compute exact solution dynamically for the current time step t_vals(j)
        step_exact = y_anal(t_vals(j));
        step_error = abs(step_exact - y_vals(j));

        % Print each step row cleanly (j-1 gives us step 0, step 1, step 2...)
        fprintf("%-6d  %-10.2f  %-10.4f  %-10.4f  %-10.4e\n", ...
                j-1, t_vals(j), step_exact, y_vals(j), step_error);
    end
end

function [iter, t_vals, y_vals, exact, err] = euler(target_X, x0, y0, h, y_prime, y_anal)
```

```
    x = x0;
    y = y0;
    iter = 0;
    t_vals = [x];
    y_vals = [y];
    iter = 0;

    while x < target_X-1e-9
        iter = iter + 1;

        y = y + h * y_prime(x,y);
        x = x + h;

        t_vals(iter+1) = x;
        y_vals(iter+1) = y;
    end

    exact = y_anal(target_X);
    err = abs(exact - y);
end
```

Output

```
[Comparison]

h         Total_Iter    Exact_Val     Approx_Val    Abs_Error
-----------------------------------------------------------------
0.50      2             1.50000       2.00000       5.00000e-01
0.25      4             1.50000       1.71482       2.14824e-01
0.10      10            1.50000       1.57967       7.96695e-02
0.01      100           1.50000       1.50764       7.64154e-03

[Detailed Table]

Table for h = 0.50
iter    t_vals      exact       y_vals      error
=================================================================
0       0.00        1.0000      1.0000      0.0000e+00
1       0.50        1.6000      2.0000      4.0000e-01
2       1.00        1.5000      2.0000      5.0000e-01

Table for h = 0.25
iter    t_vals      exact       y_vals      error
=================================================================
0       0.00        1.0000      1.0000      0.0000e+00
1       0.25        1.4118      1.5000      8.8235e-02
2       0.50        1.6000      1.7941      1.9412e-01
3       0.75        1.6000      1.8353      2.3529e-01
4       1.00        1.5000      1.7148      2.1482e-01

Table for h = 0.10
iter    t_vals      exact       y_vals      error
=================================================================
0       0.00        1.0000      1.0000      0.0000e+00
1       0.10        1.1881      1.2000      1.1881e-02
2       0.20        1.3462      1.3743      2.8104e-02
3       0.30        1.4679      1.5137      4.5819e-02
4       0.40        1.5517      1.6139      6.2148e-02
5       0.50        1.6000      1.6750      7.4984e-02
6       0.60        1.6176      1.7010      8.3338e-02
```

```
7       0.70        1.6107      1.6980      8.7219e-02
8       0.80        1.5854      1.6726      8.7280e-02
9       0.90        1.5470      1.6314      8.4451e-02
10      1.00        1.5000      1.5797      7.9669e-02

Table for h = 0.01
iter    t_vals      exact       y_vals      error
============================================================
0       0.00        1.0000      1.0000      0.0000e+00
1       0.01        1.0199      1.0200      1.0199e-04
2       0.02        1.0396      1.0398      2.0985e-04
3       0.03        1.0590      1.0594      3.2341e-04
4       0.04        1.0783      1.0787      4.4248e-04
5       0.05        1.0973      1.0978      5.6684e-04
6       0.06        1.1160      1.1167      6.9627e-04
7       0.07        1.1344      1.1353      8.3055e-04
8       0.08        1.1526      1.1536      9.6942e-04
9       0.09        1.1705      1.1716      1.1126e-03
10      0.10        1.1881      1.1894      1.2599e-03
11      0.11        1.2054      1.2068      1.4110e-03
12      0.12        1.2224      1.2240      1.5657e-03
13      0.13        1.2391      1.2408      1.7235e-03
14      0.14        1.2554      1.2573      1.8843e-03
15      0.15        1.2714      1.2734      2.0478e-03
16      0.16        1.2871      1.2893      2.2136e-03
17      0.17        1.3024      1.3047      2.3814e-03
18      0.18        1.3173      1.3199      2.5510e-03
19      0.19        1.3319      1.3346      2.7221e-03
20      0.20        1.3462      1.3490      2.8942e-03
21      0.21        1.3600      1.3631      3.0672e-03
22      0.22        1.3735      1.3768      3.2407e-03
23      0.23        1.3866      1.3901      3.4145e-03
24      0.24        1.3994      1.4030      3.5881e-03
25      0.25        1.4118      1.4155      3.7615e-03
26      0.26        1.4238      1.4277      3.9342e-03
27      0.27        1.4354      1.4395      4.1060e-03
28      0.28        1.4466      1.4509      4.2767e-03
29      0.29        1.4574      1.4619      4.4459e-03
30      0.30        1.4679      1.4725      4.6135e-03
31      0.31        1.4780      1.4827      4.7792e-03
32      0.32        1.4877      1.4926      4.9427e-03
33      0.33        1.4970      1.5021      5.1039e-03
34      0.34        1.5059      1.5112      5.2625e-03
35      0.35        1.5145      1.5199      5.4184e-03
36      0.36        1.5227      1.5282      5.5713e-03
37      0.37        1.5305      1.5362      5.7211e-03
38      0.38        1.5379      1.5438      5.8676e-03
39      0.39        1.5450      1.5510      6.0106e-03
40      0.40        1.5517      1.5579      6.1501e-03
41      0.41        1.5581      1.5644      6.2859e-03
42      0.42        1.5641      1.5705      6.4178e-03
43      0.43        1.5698      1.5763      6.5459e-03
44      0.44        1.5751      1.5817      6.6699e-03
45      0.45        1.5800      1.5868      6.7897e-03
46      0.46        1.5847      1.5916      6.9054e-03
47      0.47        1.5890      1.5960      7.0169e-03
48      0.48        1.5930      1.6001      7.1240e-03
49      0.49        1.5966      1.6039      7.2268e-03
50      0.50        1.6000      1.6073      7.3252e-03
51      0.51        1.6030      1.6105      7.4192e-03
52      0.52        1.6058      1.6133      7.5089e-03
53      0.53        1.6082      1.6158      7.5940e-03
```

54	0.54	1.6104	1.6181	7.6748e-03
55	0.55	1.6123	1.6200	7.7512e-03
56	0.56	1.6139	1.6217	7.8231e-03
57	0.57	1.6152	1.6231	7.8907e-03
58	0.58	1.6163	1.6242	7.9540e-03
59	0.59	1.6171	1.6251	8.0130e-03
60	0.60	1.6176	1.6257	8.0677e-03
61	0.61	1.6180	1.6261	8.1182e-03
62	0.62	1.6180	1.6262	8.1646e-03
63	0.63	1.6179	1.6261	8.2069e-03
64	0.64	1.6175	1.6257	8.2452e-03
65	0.65	1.6169	1.6252	8.2795e-03
66	0.66	1.6160	1.6244	8.3099e-03
67	0.67	1.6150	1.6234	8.3365e-03
68	0.68	1.6138	1.6221	8.3594e-03
69	0.69	1.6124	1.6207	8.3786e-03
70	0.70	1.6107	1.6191	8.3943e-03
71	0.71	1.6089	1.6173	8.4065e-03
72	0.72	1.6070	1.6154	8.4152e-03
73	0.73	1.6048	1.6132	8.4207e-03
74	0.74	1.6025	1.6109	8.4229e-03
75	0.75	1.6000	1.6084	8.4220e-03
76	0.76	1.5974	1.6058	8.4181e-03
77	0.77	1.5946	1.6030	8.4112e-03
78	0.78	1.5916	1.6000	8.4015e-03
79	0.79	1.5886	1.5970	8.3890e-03
80	0.80	1.5854	1.5937	8.3738e-03
81	0.81	1.5820	1.5904	8.3560e-03
82	0.82	1.5786	1.5869	8.3357e-03
83	0.83	1.5750	1.5833	8.3131e-03
84	0.84	1.5713	1.5796	8.2881e-03
85	0.85	1.5675	1.5757	8.2608e-03
86	0.86	1.5636	1.5718	8.2315e-03
87	0.87	1.5596	1.5678	8.2001e-03
88	0.88	1.5555	1.5636	8.1667e-03
89	0.89	1.5513	1.5594	8.1314e-03
90	0.90	1.5470	1.5551	8.0944e-03
91	0.91	1.5426	1.5506	8.0556e-03
92	0.92	1.5381	1.5461	8.0151e-03
93	0.93	1.5336	1.5416	7.9731e-03
94	0.94	1.5290	1.5369	7.9297e-03
95	0.95	1.5243	1.5322	7.8848e-03
96	0.96	1.5196	1.5274	7.8385e-03
97	0.97	1.5148	1.5226	7.7910e-03
98	0.98	1.5099	1.5176	7.7423e-03
99	0.99	1.5050	1.5127	7.6925e-03
100	1.00	1.5000	1.5076	7.6415e-03

Code — A6Q3.m

```
clc; close all; clear all;

% Differential equation & Analytical exact solution handles
f = @(t,y) (y/t - (y/t)^2);
f_anal = @(t) (t ./ (1 + log(t)));

% Parameters
t0 = 1;
t_end = 2;
y0 = 1;
h = 0.1;
```

```
% Define time vector for the steps
t_steps = t0:h:t_end;
N = length(t_steps);

%% ============================================================
% a) ODE45 Solver (Evaluated at specific t steps)
%% ============================================================
[x_ode45, y_ode45] = ode45(f, t_steps, y0);

%% ============================================================
% b) Euler's Method
%% ============================================================
y_euler = zeros(N, 1);
y_euler(1) = y0;
for i = 1:N-1
    y_euler(i+1) = y_euler(i) + h * f(t_steps(i), y_euler(i));
end

%% ============================================================
% c) RK-2 Method (Heun's Method / Modified Euler)
%% ============================================================
y_rk2 = zeros(N, 1);
y_rk2(1) = y0;
for i = 1:N-1
    k1 = f(t_steps(i), y_rk2(i));
    k2 = f(t_steps(i) + h, y_rk2(i) + h * k1);
    y_rk2(i+1) = y_rk2(i) + (h / 2) * (k1 + k2);
end

%% ============================================================
% d) RK-4 Method (FIXED: Changed [] to () indexing)
%% ============================================================
y_rk4 = zeros(N, 1);
y_rk4(1) = y0;
for i = 1:N-1
    k1 = f(t_steps(i), y_rk4(i));
    k2 = f(t_steps(i) + h/2, y_rk4(i) + h*k1/2);
    k3 = f(t_steps(i) + h/2, y_rk4(i) + h*k2/2);
    k4 = f(t_steps(i) + h, y_rk4(i) + h*k3);
    y_rk4(i+1) = y_rk4(i) + (h / 6) * (k1 + 2*k2 + 2*k3 + k4);
end

%% ============================================================
% Generate Data Table
%% ============================================================
% Calculate Exact Values
y_exact = f_anal(t_steps)';

% Display table with exact requested headings
fprintf('\n%-5s %-10s %-10s %-10s %-10s %-10s\n', 't', 'Exact', 'ODE45', 'Euler', 'RK2', 'RK4');
fprintf('%s\n', repmat('-', 1, 60));
for i = 1:N
    % FIXED: Changed t_steps[i] to t_steps(i)
    fprintf('%-5.1f %-10.6f %-10.6f %-10.6f %-10.6f %-10.6f\n', ...
        t_steps(i), y_exact(i), y_ode45(i), y_euler(i), y_rk2(i), y_rk4(i));
end

%% ============================================================
% Plotting Results
```

```
%% ============================================================
figure('Position', [100, 100, 900, 600]);
hold on; grid on;

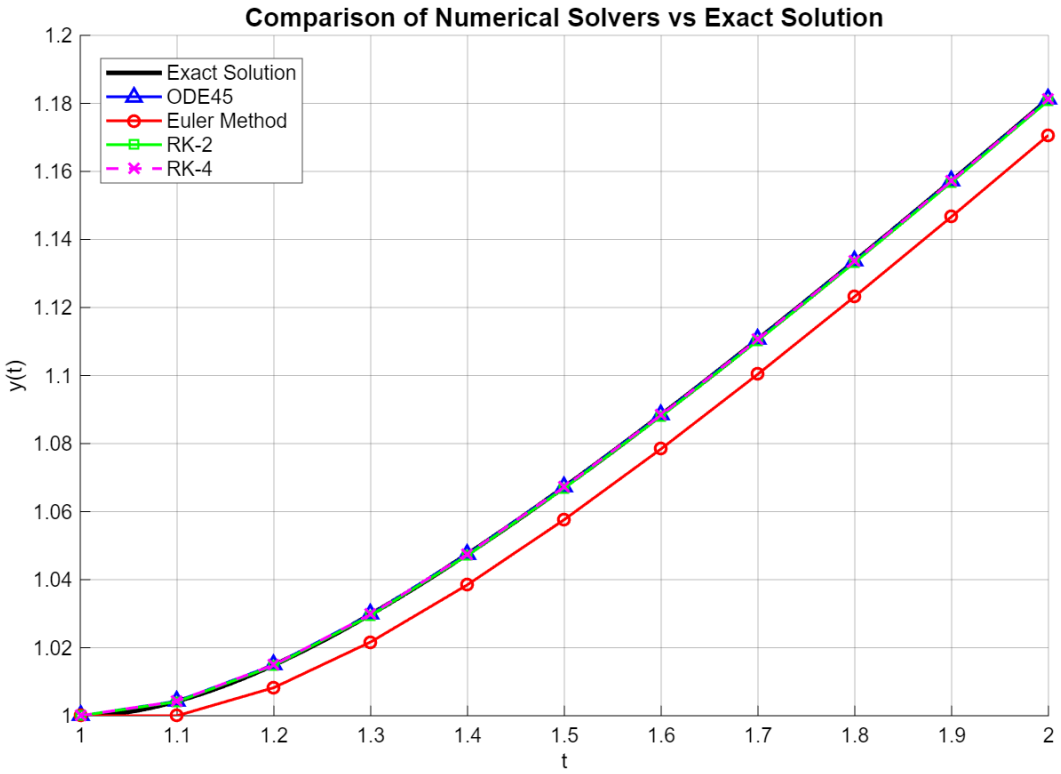
% Smooth curve for exact analytical solution
t_fine = t0:0.001:t_end;
plot(t_fine, f_anal(t_fine), 'k-', 'LineWidth', 2.5, 'DisplayName', 'Exact Solution');

% Plot approximations
plot(t_steps, y_ode45, 'b-^', 'LineWidth', 1.5, 'MarkerSize', 7, 'DisplayName', 'ODE45');
plot(t_steps, y_euler, 'r-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Euler Method');
plot(t_steps, y_rk2, 'g-s', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'RK-2');
plot(t_steps, y_rk4, 'm--x', 'LineWidth', 1.5, 'MarkerSize', 8, 'DisplayName', 'RK-4');

xlabel('t', 'FontSize', 12);
ylabel('y(t)', 'FontSize', 12);
title('Comparison of Numerical Solvers vs Exact Solution', 'FontSize', 14);
legend('Location', 'best', 'FontSize', 11);
hold off;
```

Output

```
t     Exact      ODE45      Euler      RK2        RK4
------------------------------------------------------------
1.0   1.000000   1.000000   1.000000   1.000000   1.000000
1.1   1.004282   1.004282   1.000000   1.004132   1.004282
1.2   1.014952   1.014952   1.008264   1.014714   1.014952
1.3   1.029814   1.029814   1.021689   1.029520   1.029813
1.4   1.047534   1.047534   1.038515   1.047204   1.047534
1.5   1.067262   1.067262   1.057668   1.066909   1.067262
1.6   1.088433   1.088433   1.078461   1.088064   1.088432
1.7   1.110655   1.110655   1.100432   1.110275   1.110655
1.8   1.133654   1.133654   1.123262   1.133266   1.133653
1.9   1.157228   1.157228   1.146724   1.156835   1.157228
2.0   1.181232   1.181232   1.170652   1.180834   1.181232
```



Code — A6Q4.m

```
clc; close all; clear all;

% Function definitions
f_prime = @(t,y) (y/t - (y/t).^2);
f_anal = @(t) (t ./ (1 + log(t)));

x0 = 1;
y0 = 1;
x = 2;
y = 1;
h = 0.1;
t_steps = [1:h:2];
N = length(t_steps);

%=====================================================
exact_sol = f_anal(2);

%=====================================================
% Solve using ODE45
[t_ode, y_ode] = ode45(f_prime, [1:h:2], y0);
error_ode = abs(exact_sol - y_ode);
% Solve using Oiled up method
[iter, x_history, y_history, error] = euler(x, x0, y0, h, f_prime, f_anal); %forgot to pass
functions as arguments
% Solve using the RK-2 method
[y_rk2, error_rk2] = RK_2(N, t_steps, y0, h, f_anal, f_prime);
% Solve using the RK-4 method
[y_rk4, error_rk4] = RK_4(N, t_steps, y0, h, f_anal, f_prime);
```

```
%=========================================================
% Display Result
fprintf("%-6s %-12s %-14s %-14s %-14s %-14s\n", ...
    "t", "Exact", "Error Ode45", "Error Euler", "Error Rk2", "Error Rk4");
for i = 1:N
    fprintf("%-6.1f %-12.6f %-14.6e %-14.6e %-14.6e %-14.6e\n", ...
        t_steps(i), f_anal(t_steps(i)), error_ode(i), error(i), error_rk2(i), error_rk4(i));
end

%=========================================================
% Oiled Up Method
function [iter, x_history, y_history, error] = euler(x_target, x0, y0, h, f_prime, f_anal)
    x_history = x0;
    y_history = y0;
    iter = 0;
    x = x0;
    y = y0;
    error = abs(f_anal(x0) - y0);



    while x < x_target - 1e-9
        iter = iter + 1;
        y = y + h * f_prime(x,y);
        x = x + h;

        exact = f_anal(x);
        x_history(iter + 1) = x;
        y_history(iter + 1) = y;
        error(iter + 1) = abs(exact - y);
    end
end

%=========================================================
% RK-2 Method
function [y_rk2, error_rk2] = RK_2(N, t_steps, y0, h, f_anal, f_prime)
    y_rk2 = zeros(N, 1);
    y_rk2(1) = y0;
    error_rk2(1) = abs(y_rk2(1) - f_anal(t_steps(1)));

    for i = 1:length(t_steps)-1
        exact = f_anal(t_steps(i+1));

        k1 = f_prime(t_steps(i), y_rk2(i));
        k2 = f_prime(t_steps(i) + h, y_rk2(i) + h * k1);
        y_rk2(i+1) = y_rk2(i) + (h/2) * (k1 + k2);

        error_rk2(i + 1) = abs(y_rk2(i+1) - exact);
    end
end

%=========================================================
% RK-4 Method
function [y_rk4, error_rk4] = RK_4(N, t_steps, y0, h, f_anal, f_prime)
    y_rk4 = zeros(N, 1);
    y_rk4(1) = y0;
    error_rk4 = abs(y_rk4 - t_steps(1));

    for i = 1:length(t_steps)-1
        exact = f_anal(t_steps(i+1));
```

```
        k1 = h * f_prime(t_steps(i), y_rk4(i));
        k2 = h * f_prime(t_steps(i) + h/2, y_rk4(i) + k1/2);
        k3 = h * f_prime(t_steps(i) + h/2, y_rk4(i) + k2/2);
        k4 = h * f_prime(t_steps(i) + h, y_rk4(i) + k3);

        y_rk4(i) = y_rk4(i) + h * 1/6 * (k1 + 2*k2 + 2*k3 + k4);
        error_rk4(i+1) = abs(exact - y_rk4(i+1));
    end
end
```

Output

t	Exact	Error Ode45	Error Euler	Error Rk2	Error Rk4
1.0	1.000000	1.812322e-01	0.000000e+00	0.000000e+00	0.000000e+00
1.1	1.004282	1.769505e-01	4.281728e-03	1.494965e-04	1.004282e+00
1.2	1.014952	1.662799e-01	6.687851e-03	2.386397e-04	1.014952e+00
1.3	1.029814	1.514185e-01	8.124217e-03	2.939972e-04	1.029814e+00
1.4	1.047534	1.336983e-01	9.019185e-03	3.295487e-04	1.047534e+00
1.5	1.067262	1.139699e-01	9.594162e-03	3.530391e-04	1.067262e+00
1.6	1.088433	9.279953e-02	9.971593e-03	3.689534e-04	1.088433e+00
1.7	1.110655	7.057717e-02	1.022289e-02	3.799877e-04	1.110655e+00
1.8	1.133654	4.757866e-02	1.039151e-02	3.878155e-04	1.133654e+00
1.9	1.157228	2.400379e-02	1.050484e-02	3.935040e-04	1.157228e+00
2.0	1.181232	3.339835e-10	1.058065e-02	3.977492e-04	1.181232e+00

Assignment 7

Code — A7Q1.m

```
clc; close all; clear all;
%=================== a ================================
disp('================= a ====================')
y_2 = f(-2);
fprintf("y(-2) = %f\n", y_2)

y_4 = f(4);
fprintf("y(4) = %f\n", y_4)
disp('========================================')

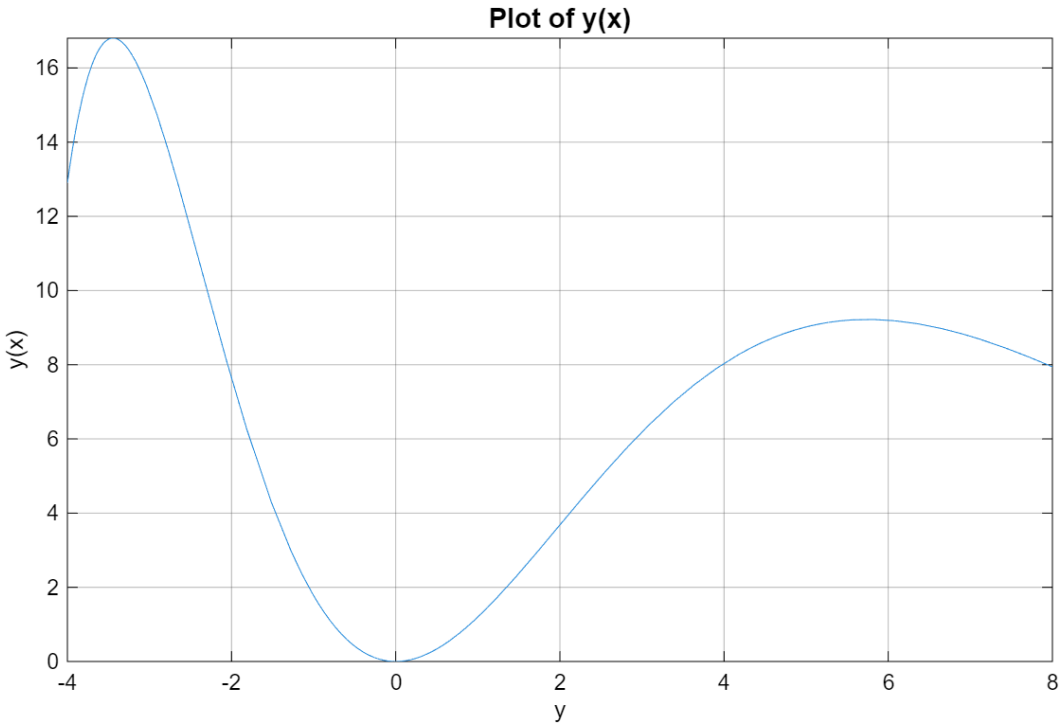
%=================== b ================================
fplot(@(x) f(x),[-4, 8])
title("Plot of y(x)")
xlabel("y")
ylabel("y(x)")
grid on;



function y = f(x)
    y = 0.6.*x.^3*exp(-0.47.*x) + 1.5.*x.^2*exp(-0.6.*x);
end
```

Output

```
================= a ====================
y(-2) = 7.632791
y(4) = 8.036691
========================================
Warning: Function behaves unexpectedly on array inputs. To improve performance, properly
vectorize your function to return an output with the same size and shape as the input
arguments.
> In matlab.graphics.function.FunctionLine>getFunction
In matlab.graphics.function/FunctionLine/updateFunction
In matlab.graphics.function/FunctionLine
In fplot>singleFplot (line 268)
In fplot>@(f)singleFplot(cax,{f},limits,extraOpts,args) (line 223)
In fplot>vectorizeFplot (line 223)
In fplot (line 186)
In A7Q1 (line 12)
In run (line 112)
In wrapper (line 6)
In run (line 112)
```



Code — A7Q2.m

```
clc; close all; clear all;

x_data = [0.9, 1.5, 3, 4, 6, 8, 9.5];
y_data = [0.9, 1.5, 2.5, 5.1, 4.5, 4.9, 6.3];

% 1. Create a smooth x array that spans your ENTIRE dataset (from 0.9 to 9.5)
x_fit = linspace(min(x_data), max(x_data), 200);

% 2. Plot the raw data exactly ONCE
figure;
plot(x_data, y_data, 'ko', 'MarkerSize', 8, 'LineWidth', 2, 'MarkerFaceColor', 'k');
hold on; % Keep the window open so everything layers together

% 3. Create a cell array to dynamically build the legend entries
legend_labels = {'Original Data'};

% 4. Loop through degrees 1 to 6 to fit, evaluate, and plot each curve
for i = 1:6
    p = polyfit(x_data, y_data, i);
    y_fit = polyval(p, x_fit);

    plot(x_fit, y_fit, 'LineWidth', 2);

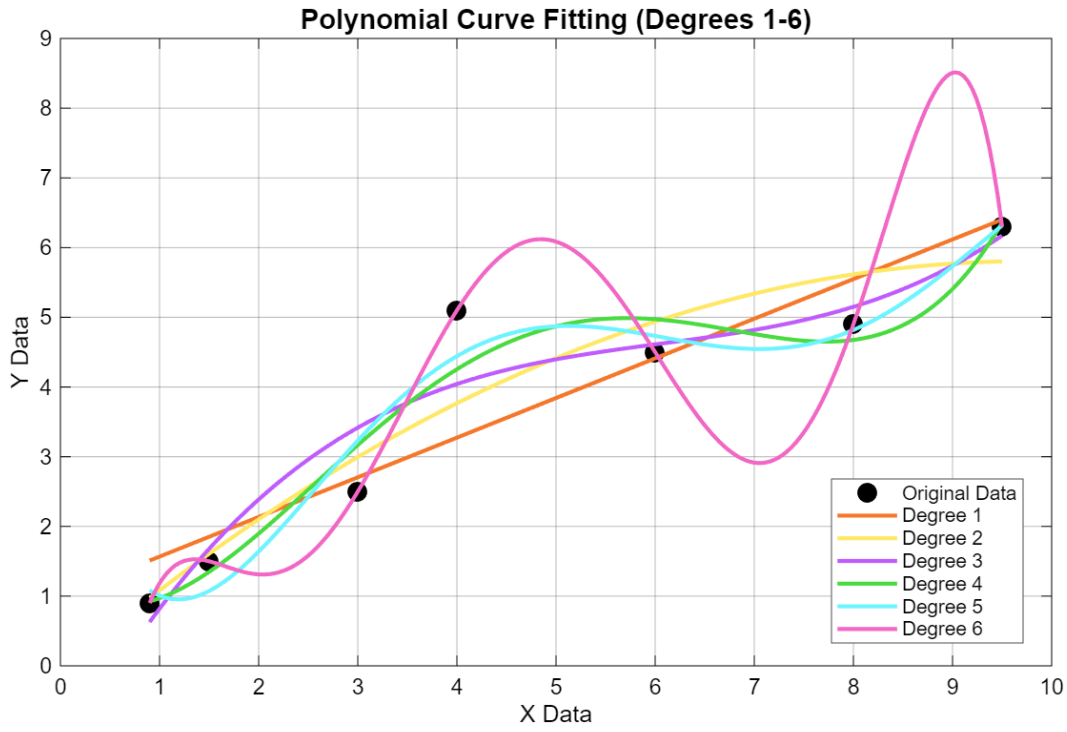
    % Append the degree label to our legend list
    legend_labels{end+1} = ['Degree ' num2str(i)];
end

% 5. Apply graph styles AFTER all data has been plotted
grid on;
```

```
xlabel('X Data');
ylabel('Y Data');
title('Polynomial Curve Fitting (Degrees 1-6)');
legend(legend_labels, 'Location', 'best');

hold off; % Release the plot
```

Output



Code — A7Q3.m

```
% Given data
x = [-5, -4, -1, 1, 4, 6, 9, 10];
y = [12, 10, 6, 2, -3, -6, -11, -12];

% a) Determine coefficients m and b using polyfit (degree 1)
p = polyfit(x, y, 1);
m = p(1);
b = p(2);

fprintf('Slope (m) = %.4f\n', m);
fprintf('Y-Intercept (b) = %.4f\n', b);
fprintf('Best-fit equation: y = %.4fx + %.4f\n', m, b);

% b) Make a plot showing the function and the data points
figure;
scatter(x, y, 60, 'blue', 'filled', 'DisplayName', 'Data Points');
hold on;

% Generate smooth x-values for plotting the line
```

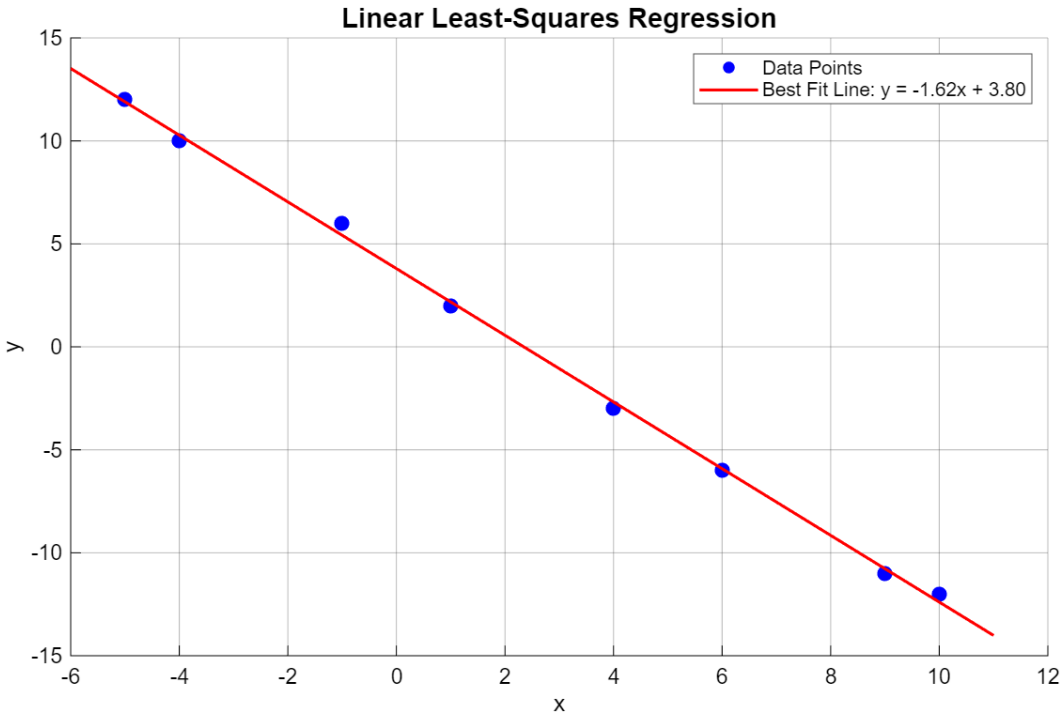
```
x_line = linspace(min(x) - 1, max(x) + 1, 100);
y_line = m * x_line + b;

plot(x_line, y_line, 'r-', 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Best Fit Line: y = %.2fx + %.2f', m, b));

% Formatting the plot
grid on;
xlabel('x');
ylabel('y');
title('Linear Least-Squares Regression');
legend('Location', 'northeast');
hold off;
```

Output

```
Slope (m) = -1.6195
Y-Intercept (b) = 3.7987
Best-fit equation: y = -1.6195x + 3.7987
```



Code — A7Q4.m

```
clc; close all; clear all;

% Given data
t = [0, 1, 3, 4, 6, 7, 9];
NB = [500, 600, 1000, 1400, 2100, 2700, 4100];
t_est = 5.0;
t_dense = linspace(min(t), max(t), 200);

%% ========================================
% Part a: Exponential Fit (NB = N * exp(alpha * t))
```

```
% ========================================
% Linearize via natural logarithm: ln(NB) = ln(N) + alpha * t
p = polyfit(t, log(NB), 1);
alpha = p(1);
N_coeff = exp(p(2));

% Estimate at t = 5h
NB_exp_5 = N_coeff * exp(alpha * t_est);

fprintf('--- Part a: Exponential Fit ---\n');
fprintf('Equation: NB = %.2f * exp(%.4f * t)\n', N_coeff, alpha);
fprintf('Estimated NB at t = 5h: %.2f\n\n', NB_exp_5);

% Plot for Part a
figure('Name', 'Exponential Fit', 'Position', [100, 100, 800, 500]);
scatter(t, NB, 60, 'black', 'filled', 'DisplayName', 'Data Points');
hold on;
plot(t_dense, N_coeff * exp(alpha * t_dense), 'r-', 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Exp Fit: N_B = %.2f e^{%.3ft}', N_coeff, alpha));
scatter(t_est, NB_exp_5, 100, 'magenta', 'p', 'filled', ...
    'DisplayName', sprintf('Estimate at 5h: %.2f', NB_exp_5));
xlabel('Time t (h)');
ylabel('Bacteria Count N_B');
title('a) Exponential Curve Fitting');
legend('Location', 'northwest');
grid on;
hold off;



%% ========================================
% Part b: Interpolation Methods
% ========================================
% 1. Linear Interpolation
NB_linear_5 = interp1(t, NB, t_est, 'linear');

% 2. Cubic Spline Interpolation (using 'spline')
NB_spline_5 = spline(t, NB, t_est);

% 3. PCHIP Interpolation (Shape-preserving piecewise cubic)
NB_pchip_5 = pchip(t, NB, t_est);

fprintf('--- Part b: Interpolation Estimates at t = 5h ---\n');
fprintf('Linear Interpolation: %.2f\n', NB_linear_5);
fprintf('Cubic Spline Interpolation: %.2f\n', NB_spline_5);
fprintf('PCHIP Interpolation: %.2f\n', NB_pchip_5);

% Evaluate dense curves for plotting
y_linear =     (t, NB, t_dense, 'linear');
y_spline = spline(t, NB, t_dense);
y_pchip = pchip(t, NB, t_dense);

% Plot for Part b
figure('Name', 'Interpolation Methods', 'Position', [150, 150, 800, 500]);
scatter(t, NB, 60, 'black', 'filled', 'DisplayName', 'Data Points');
hold on;
plot(t_dense, y_linear, 'b--', 'LineWidth', 1.5, 'DisplayName', 'Linear Interpolation');
plot(t_dense, y_spline, 'g-', 'LineWidth', 1.5, 'DisplayName', 'Cubic Spline');
plot(t_dense, y_pchip, 'Color', [0.85, 0.33, 0.1], 'LineStyle', '-.', 'LineWidth', 1.5,
'DisplayName', 'PCHIP Interpolation');
```

```
% Mark estimates at t = 5h
scatter(t_est, NB_linear_5, 80, 'blue', 'o', 'filled', 'HandleVisibility', 'off');
scatter(t_est, NB_spline_5, 80, 'green', 's', 'filled', 'HandleVisibility', 'off');
scatter(t_est, NB_pchip_5, 80, [0.85, 0.33, 0.1], '^', 'filled', 'HandleVisibility', 'off');

xlabel('Time t (h)');
ylabel('Bacteria Count N_B');
title('b) Interpolation Methods (Linear, Spline, PCHIP)');
legend('Location', 'northwest');
grid on;
hold off;
```

Output

```
ERROR: Error: File: C:\Users\user\Desktop\Matlab-and-R\ASSIGNMENT 7\A7Q4.m Line: 58 Column: 15
Unexpected ',' in parentheses. When creating a matrix, use square brackets [].
```

Code — A7Q5.m

```
clc; close all; clear all;
% Define time vector from 0 to 20 s with high resolution
t = linspace(0, 20, 1000);

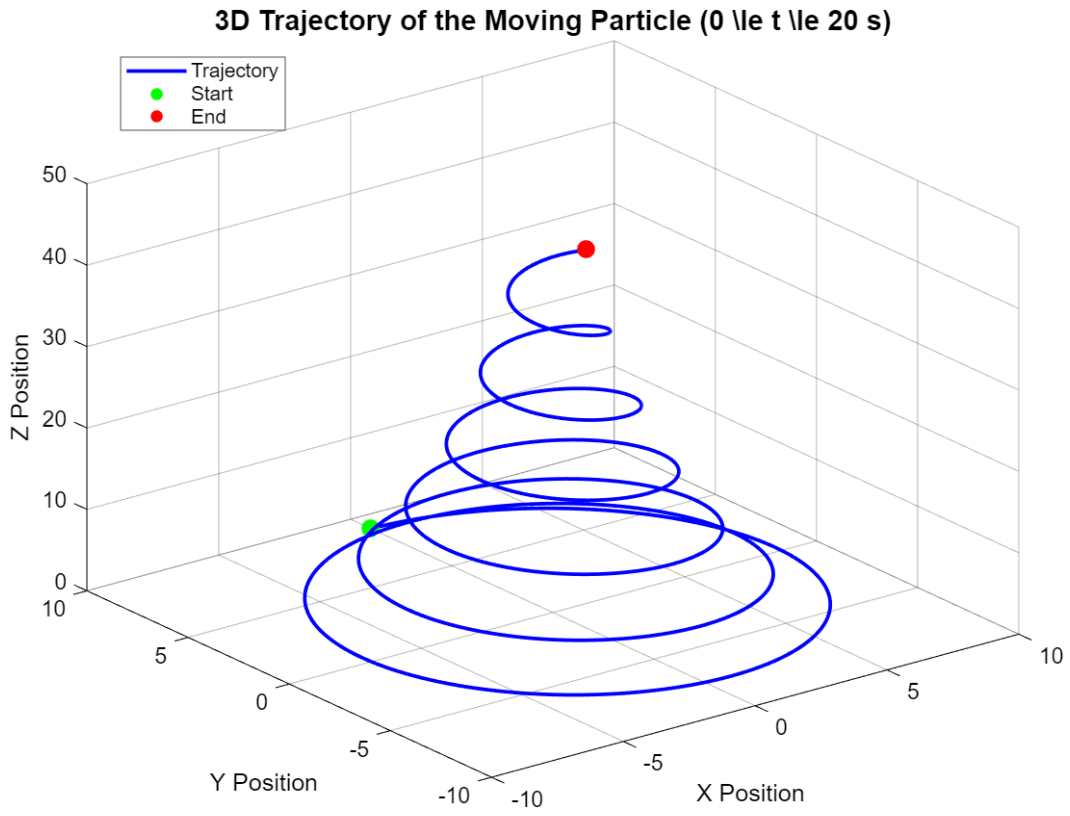
% Calculate position coordinates as a function of time
x = 0.01 .* (30 - t).^2 .* sin(2 * t);
y = 0.01 .* (30 - t).^2 .* cos(2 * t);
z = 0.5 .* t.^1.5;

% Create 3D figure
figure('Name', 'Particle Trajectory', 'Position', [100, 100, 800, 600]);
plot3(x, y, z, 'LineWidth', 1.8, 'Color', 'b');
hold on;

% Mark the start (t = 0 s) and end (t = 20 s) positions
scatter3(x(1), y(1), z(1), 80, 'green', 'filled', 'DisplayName', 'Start (t = 0 s)');
scatter3(x(end), y(end), z(end), 80, 'red', 'filled', 'DisplayName', 'End (t = 20 s)');

% Format axes and labels
grid on;
xlabel('X Position');
ylabel('Y Position');
zlabel('Z Position');
title('3D Trajectory of the Moving Particle (0 \le t \le 20 s)');
legend('Trajectory', 'Start', 'End', 'Location', 'best');
view(3); % Set standard 3D view angle
hold off;
```

Output



Code — A7Q6.m

```
clc; close all; clear all;

% Data setup: Rows = Years (2020-2023), Columns = Cities (Dhaka, Sylhet, Chattogram, Rajshahi,
Khulna)
years = [2020; 2021; 2022; 2023];
rainfall_matrix = [
    850, 920, 780, 690, 860;
    900, 890, 820, 720, 880;
    870, 940, 790, 700, 890;
    910, 930, 810, 740, 870
];
city_names = {'Dhaka', 'Sylhet', 'Chattogram', 'Rajshahi', 'Khulna'};

%% a) Grouped Bar Chart
figure('Name', 'Grouped Bar Chart - Annual Rainfall', 'Position', [100, 100, 850, 500]);
bar(years, rainfall_matrix, 'grouped');
title('Annual Rainfall of Cities in Bangladesh (2020-2023)', 'FontSize', 14, 'FontWeight',
'bold');
xlabel('Year', 'FontSize', 12);
ylabel('Rainfall (mm)', 'FontSize', 12);
legend(city_names, 'Location', 'northeastoutside');
grid on;

%% b) 3D Pie Chart of Total Rainfall Contribution
% Calculate total rainfall across all 4 years for each city
total_rainfall = sum(rainfall_matrix, 1);

figure('Name', '3D Pie Chart - Rainfall Contribution', 'Position', [150, 150, 750, 650]);
```

```
pie3(total_rainfall);
title('City Contribution to Total Rainfall (2020-2023)', 'FontSize', 14, 'FontWeight', 'bold');
legend(city_names, 'Location', 'northeastoutside');
%==================================================================
```

Output

